

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Algoritmos y Estructuras de Datos- Sección 16026

Proyecto Final – Documentación.

Integrantes:

Roberto Agustín Mejía Collazos

Miguel Ángel Velásquez Ysuiza

Manuel Ángel Pecho Santos

Ivan Christian Sulca Palacios

Lima - Peru

ÍNDICE

Contenido

CAPÍTULO

Contenido

CAPÍTULO 2
1.1. DEFINICIÓN DEL PROBLEMA 3
1.1.1 DESCRIPCIÓN DEL PROBLEMA 3
1.2. DEFINICIÓN DE OBJETIVOS 3
1.2.1 OBJETIVOS DEL SISTEMA 3
1.3. ALCANCE 4
1.3.1 DESCRIPCIÓN DE LOS FORMULARIOS 4
1.3.2 DESCRIPCIÓN DE LOS ARCHIVOS 5
CAPITULO 2 – DISEÑO DE LA APLICACIÓN 6
2.1 ESTRUCTURA GENERAL DEL SISTEMA 6
2.1.1 DISEÑO DE LAS CLASES 7
2.1.2 DISEÑO DE LAS ESTRUCTURAS DE DATOS 7
3.1 CONCLUSIONES 36
4.1 RECOMENDACIONES 36
5.1 BIBLIOGRAFÍA 37
6.1 HISTORIAL DE VERSIONES 38

Proyecto Final – Documentacion.

1.1. DEFINICIÓN DEL PROBLEMA

1.1.1 DESCRIPCIÓN DEL PROBLEMA

El sistema de gestión de elecciones en España enfrenta varios desafíos en cuanto a la automatización y precisión de los procesos electorales. Actualmente, los procedimientos tradicionales, como el registro de votantes, el conteo de votos y la generación de resultados, son gestionados de manera manual, lo que aumenta el riesgo de errores humanos y retrasa los tiempos de procesamiento. Además, debido a la gran cantidad de datos involucrados en cada elección, como el registro de votantes, los candidatos y los resultados por mesa y distrito, el manejo de esta información se vuelve cada vez más complejo y propenso a inconsistencias.

Las áreas principales que componen el sistema electoral incluyen el registro de votantes, la gestión de los candidatos y partidos políticos, el seguimiento del conteo de votos, y la generación de informes de resultados. Estas áreas requieren un uso intensivo de recursos y personal, lo que genera costos operativos elevados y limita la capacidad de realizar procesos de verificación y auditoría de manera eficiente.

Los problemas que se presentan en este contexto incluyen:

- Procesos manuales: La introducción manual de datos y el conteo de votos conllevan una alta probabilidad de error y una gran carga de trabajo para los operadores.
- Manejo ineficiente de grandes volúmenes de datos: La gestión de registros de votantes, partidos y votos requiere un manejo estructurado y automatizado para evitar la pérdida de información y mejorar la capacidad de consulta y análisis.
- Falta de precisión en los resultados: La generación de resultados manuales puede ser imprecisa, lo que afecta la confianza del público en el proceso electoral.

Este proyecto tiene como objetivo resolver estas deficiencias mediante el desarrollo de una aplicación informática que automatice el proceso electoral, optimizando el registro de votantes, el conteo de votos y la generación de resultados, mejorando la eficiencia y reduciendo los márgenes de error. La implementación de un sistema automatizado permitirá no solo mejorar la precisión en el conteo de votos, sino también facilitar la gestión de grandes volúmenes de datos de manera más ágil y eficiente.

1.2. DEFINICIÓN DE OBJETIVOS

1.2.1 OBJETIVOS DEL SISTEMA

El objetivo principal del sistema es desarrollar una herramienta integral que automatice y facilite la gestión de elecciones en España, desde el registro de votantes hasta la generación de reportes finales, asegurando precisión y eficiencia en cada etapa del proceso. El sistema

permitirá importar, procesar y exportar datos en formatos estándar como CSV y PDF, optimizando el flujo de trabajo electoral y minimizando los errores humanos.

Los objetivos específicos del sistema son los siguientes:

- Automatizar el proceso electoral: Incluir la automatización del registro de votos, la actualización de los resultados y la generación de reportes detallados de forma rápida y precisa.
- Gestión flexible de datos electorales: Implementar un sistema que permita la importación y exportación de datos en formatos estándar como CSV y PDF, facilitando el intercambio de información entre distintas plataformas.
- Interfaz amigable para el usuario: Desarrollar una interfaz intuitiva que permita a los usuarios realizar consultas de votos de manera eficiente, con filtros dinámicos para obtener resultados por diferentes criterios como distrito, sección, mesa o tipo de voto.
- Garantizar precisión y transparencia: Asegurar que el sistema proporcione datos exactos y actualizados, fomentando la transparencia en los procesos electorales y ofreciendo una verificación clara de los resultados.

1.3. ALCANCE

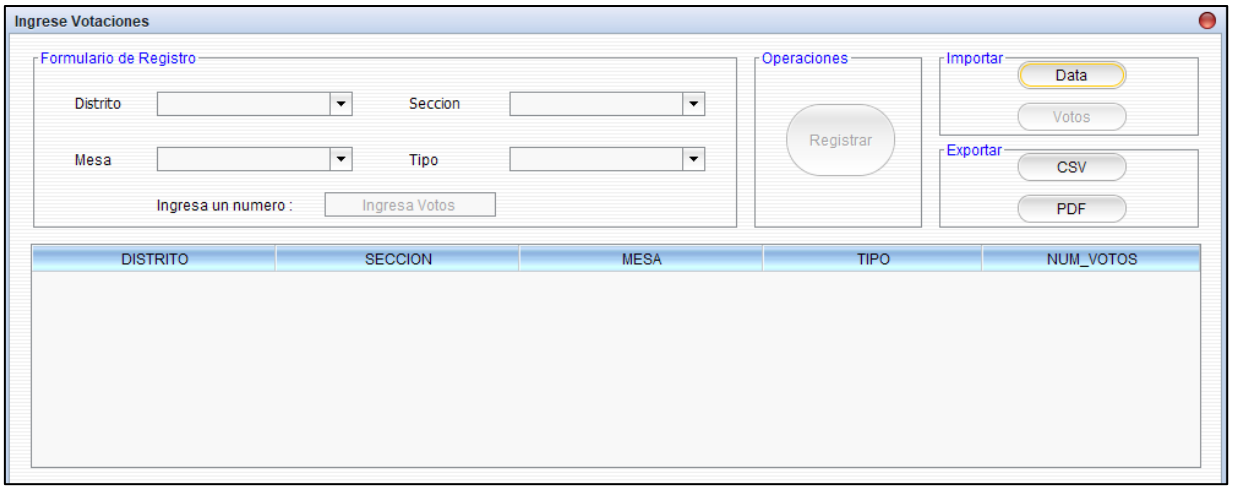
El sistema automatiza el ciclo completo de una elección en España, desde el registro de votantes hasta la generación de resultados finales. Permite la importación de datos, contabilización de votos, visualización de resultados y exportación a formatos CSV y PDF. También incluye filtros para consultas detalladas por distrito, sección, mesa y tipo de voto. El sistema optimiza el flujo electoral y asegura la precisión de los resultados.

1.3.1 DESCRIPCIÓN DE LOS FORMULARIOS

Los formularios del sistema incluyen:

Nombre del Formulario	Descripción	Usuarios
Formulario de Registro	Permite realizar las operaciones básicas de: - Registrar votos. - Consultar los votos registrados. - Modificar la información de votos. - Eliminar votos.	- Administrador del sistema - Usuario registrado
Importación de Datos	Permite importar datos de votantes y votos desde archivos externos.	- Administrador del sistema
Exportación de Resultados	Permite exportar los resultados de las elecciones en formatos CSV o PDF.	- Administrador del sistema

Control	Título	Evento	Descripción
btnRegistrar	Registrar Voto	Clic	Graba un nuevo voto en el sistema, añadiendo la información al registro de votos.
btnModificar	Modificar Voto	Clic	Actualiza los datos del voto seleccionado en el registro.
btnEliminar	Eliminar Voto	Clic	Elimina un voto previamente registrado en el sistema.
btnImportar	Importar Datos	Clic	Permite importar los datos de votantes o votos desde un archivo CSV o PDF.
btnExportar	Exportar Resultados	Clic	Exporta los resultados de las elecciones en formato CSV o PDF para su análisis o presentación.



1.3.2 DESCRIPCIÓN DE LOS ARCHIVOS

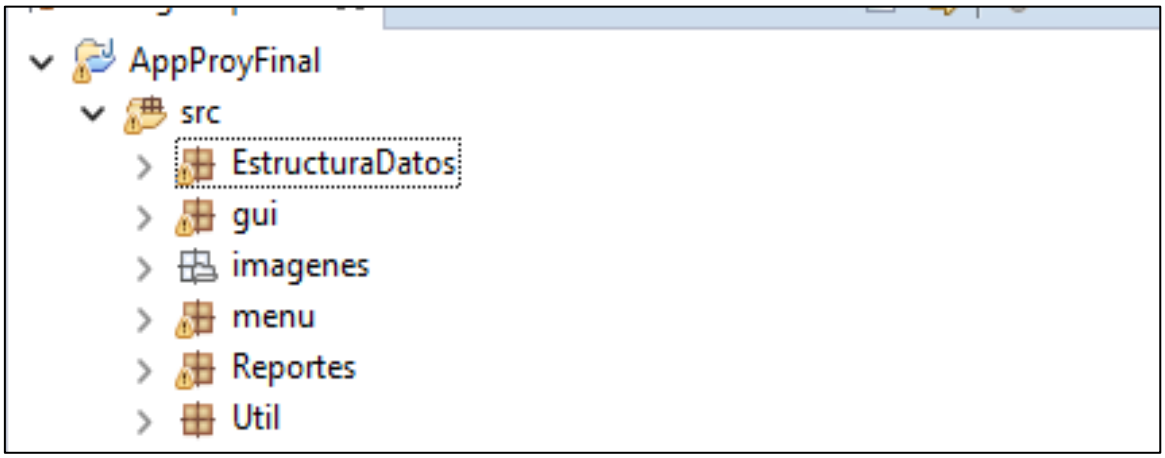
El archivo ElecGenEspania_v2.csv contiene datos detallados sobre las elecciones generales de España. En él se registran los resultados de los votos distribuidos por distrito, sección y mesa electoral, especificando el número de votos obtenidos por cada partido político o categoría de votos, como los votos válidos, nulos, o abstenciones. Las columnas incluyen el Título de las elecciones, la Fecha de las elecciones, junto con la información desglosada por Distrito, Sección y Mesa electoral, el nombre de cada Partido y el total de Num Votos obtenidos.

Columna	Descripción	Tipo de Dato
Año	Año de las elecciones (por ejemplo, 2023)	Integer (2023)

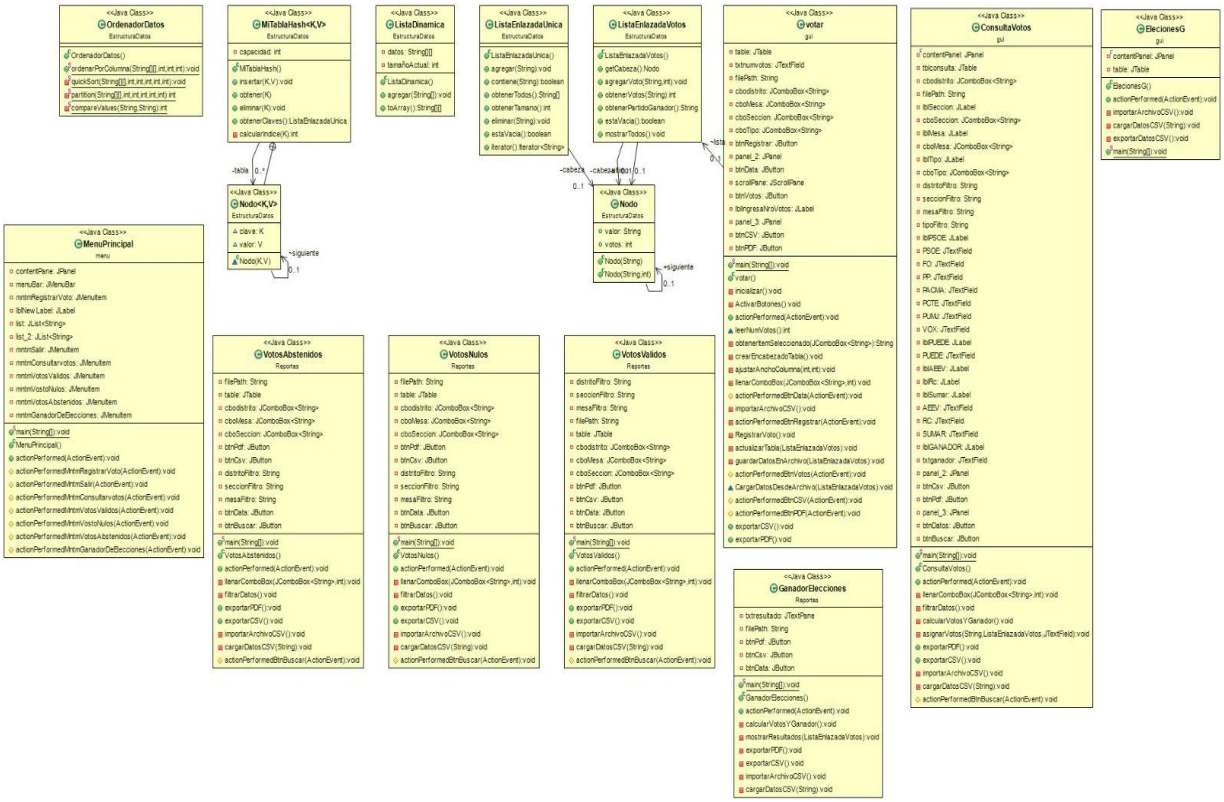
Título	Título de las elecciones (por ejemplo, "Elecciones Generales")	String (Texto)
Fecha Elecciones	Fecha de las elecciones (formato DD/MM/YYYY)	Date (DD/MM/YYYY)
Distrito	Número del distrito electoral	Integer
Sección	Número de la sección electoral	Integer
Mesa	Número de la mesa electoral (por ejemplo, "A")	String (Letra)
Partido	Nombre del partido político o categoría de votos (por ejemplo, "PSOE")	String (Texto)
Num Votos	Número de votos en esa categoría	Integer

CAPITULO 2 – DISEÑO DE LA APLICACIÓN

2.1 ESTRUCTURA GENERAL DEL SISTEMA

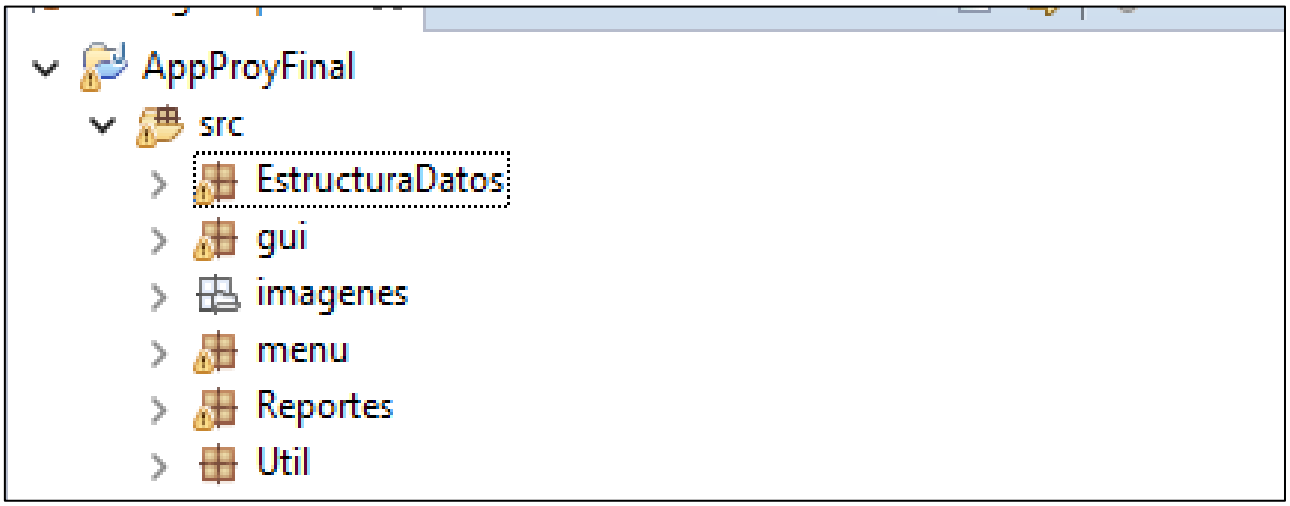


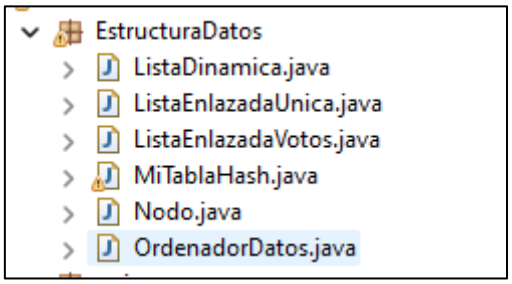
2.1.1 DISEÑO DE LAS CLASES



2.1.2 DISEÑO DE LAS ESTRUCTURAS DE DATOS

Paquete Estructura de datos:





Clases de Estructura de datos:

La clase ListaDinamica la utilizo para manejar y almacenar datos de manera flexible en mi proyecto de elecciones. Cada vez que necesito agregar un nuevo registro, como un votante o un candidato, la clase gestiona dinámicamente la capacidad de la lista, asegurándose de que no haya limitaciones en el número de elementos. Utilizo un arreglo bidimensional donde cada fila representa un registro y las columnas contienen atributos como nombre, partido y número de votos. Además, el método toArray me permite convertir los datos almacenados en un arreglo con el tamaño exacto que necesito para trabajar con ellos de manera más eficiente.

.project | MenuPrincipal.java | LogStrings.properties | ListaDinamica.java

```java
package EstructuraDatos;

public class ListaDinamica {
    private String[][] datos; // Matriz para almacenar los elementos de la lista
    private int tamañoActual; // Número actual de elementos en la lista

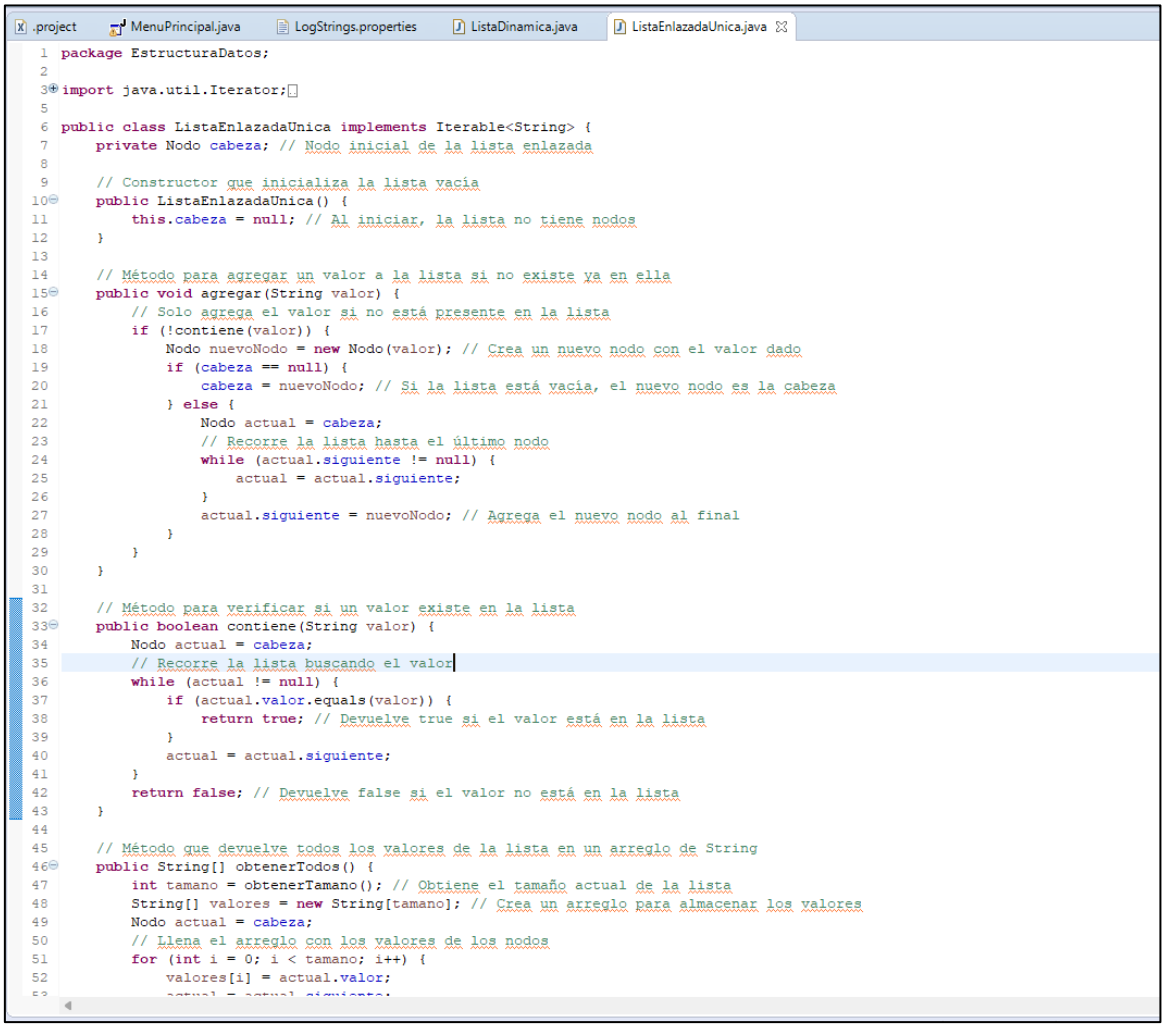
    // Constructor que inicializa la lista con un tamaño inicial de 10 filas y 8 columnas
    public ListaDinamica() {
        datos = new String[10][8]; // Arreglo bidimensional de tamaño inicial 10x8
        tamañoActual = 0; // Inicialmente no hay elementos en la lista
    }



    // Método para agregar un nuevo elemento a la lista
    public void agregar(String[] elemento) {
        // Verifica si el arreglo está lleno
        if (tamañoActual == datos.length) {
            // Si está lleno, crea un nuevo arreglo con el doble de filas y 8 columnas
            String[][] nuevoDatos = new String[datos.length * 2][8];
            // Copia los elementos existentes al nuevo arreglo
            for (int i = 0; i < datos.length; i++) {
                nuevoDatos[i] = datos[i];
            }
            // Asigna el nuevo arreglo a datos
            datos = nuevoDatos;
        }
        // Agrega el nuevo elemento en la posición actual y aumenta el tamaño
        datos[tamañoActual] = elemento;
        tamañoActual++;
    }

    // Método que devuelve los elementos añadidos en un nuevo arreglo de tamaño exacto
    public String[][] toArray() {
        // Crea un arreglo con el tamaño exacto de los elementos en la lista
        String[][] resultado = new String[tamañoActual][8];
        // Copia cada elemento del arreglo original al nuevo arreglo
        for (int i = 0; i < tamañoActual; i++) {
            resultado[i] = datos[i];
        }
        return resultado; // Devuelve el arreglo con los elementos añadidos
    }
}
```

La clase ListaEnlazadaUnica la uso para gestionar una lista de elementos donde cada elemento está vinculado al siguiente mediante un nodo. Es útil en mi proyecto de elecciones para almacenar datos de manera secuencial, como por ejemplo una lista de candidatos o votantes. La lista se mantiene dinámica, permitiendo agregar nuevos elementos solo si no existen previamente, y recorrerla de forma eficiente. La clase permite agregar valores, verificar si ya están presentes, eliminar elementos y obtener todos los valores en un arreglo. También incluye un método para obtener el tamaño de la lista y para verificar si está vacía. Utilizo un iterador para recorrer la lista de manera más estructurada y para realizar operaciones sobre los datos de forma fluida en mi aplicación.



la clase ListaEnlazadaVotos la para gestionar los votos de los partidos políticos de forma eficiente. Cada vez que se recibe un voto, la clase verifica si el partido ya está en la lista. Si es así, suma los votos al total existente; si no, crea un nuevo nodo para agregar el partido y sus votos. Además, tiene métodos para obtener los votos de un partido específico, encontrar el partido con más votos (el ganador), verificar si la lista está vacía y mostrar todos los partidos con sus votos. La estructura de la lista enlazada me permite añadir votos de manera dinámica y manejar de forma sencilla las actualizaciones y consultas de los resultados de las elecciones.

.project | MenuPrincipal.java | LogStrings.properties | ListaDinamica.java | ListaEnlazadaUnica.java | ListaEnlazadaVotos.java

```java
package EstructuraDatos;

public class ListaEnlazadaVotos {

    private Nodo cabeza;   // Nodo inicial de la lista
    private Nodo ultimo;   // Referencia al último nodo de la lista

    // Constructor que inicializa la lista vacía
    public ListaEnlazadaVotos() {
        this.cabeza = null;  // No hay nodos al inicio
        this.ultimo = null;  // No hay un nodo final tampoco
    }

    // Métodos para acceder al nodo cabeza
    public Nodo getCabeza() {
        return cabeza;
    }

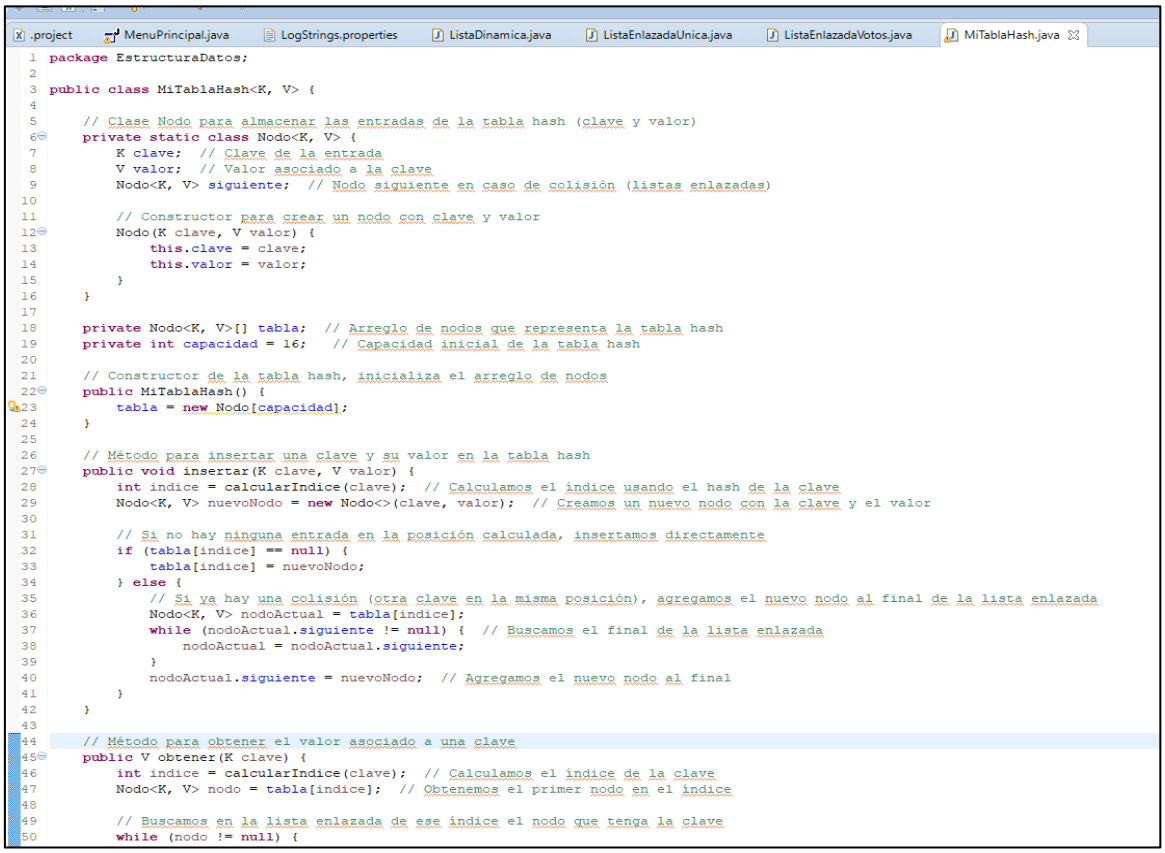

    // Agregar votos a un partido; si el partido ya existe, suma los votos
    public void agregarVoto(String partido, int votos) {
        Nodo actual = cabeza;

        // Busca si el partido ya está en la lista
        while (actual != null) {
            if (actual.valor.equals(partido)) {
                actual.votos += votos;  // Suma los votos si el partido ya existe
                return;
            }
            actual = actual.siguiente;
        }

        // Si el partido no está, se crea un nuevo nodo y se agrega al final
        Nodo nuevoNodo = new Nodo(partido, votos);  // Crea el nodo con partido y votos
        if (cabeza == null) {
            cabeza = nuevoNodo;  // La lista está vacía, así que el nuevo nodo es la cabeza
            ultimo = nuevoNodo;  // También es el último nodo
        } else {
            ultimo.siguiente = nuevoNodo;  // Agrega el nuevo nodo al final de la lista
            ultimo = nuevoNodo;            // Actualiza la referencia al último nodo
        }
    }

    // Método para obtener los votos de un partido específico
    public int obtenerVotos(String partido) {
        Nodo actual = cabeza;
        while (actual != null) {
            if (actual.valor.equals(partido)) {
                return actual.votos;  // Retorna los votos si el partido se encuentra
            }
            actual = actual.siguiente;
```

La clase MiTablaHash gestiona pares clave-valor de manera eficiente utilizando una tabla hash. Al insertar un par, calcula un índice basado en el hashCode de la clave. Si hay colisión, los nodos se enlazan en listas dentro del mismo índice. Permite insertar, obtener, eliminar pares y obtener todas las claves almacenadas. La tabla utiliza listas enlazadas para manejar las colisiones de manera dinámica.




```java
package EstructuraDatos;

public class MiTablaHash<K, V> {

    // Clase Nodo para almacenar las entradas de la tabla hash (clave y valor)
    private static class Nodo<K, V> {
        K clave;  // Clave de la entrada
        V valor;  // Valor asociado a la clave
        Nodo<K, V> siguiente;  // Nodo siguiente en caso de colisión (listas enlazadas)

        // Constructor para crear un nodo con clave y valor
        Nodo(K clave, V valor) {
            this.clave = clave;
            this.valor = valor;
        }
    }

    private Nodo<K, V>[] tabla;  // Arreglo de nodos que representa la tabla hash
    private int capacidad = 16;   // Capacidad inicial de la tabla hash

    // Constructor de la tabla hash, inicializa el arreglo de nodos
    public MiTablaHash() {
        tabla = new Nodo[capacidad];
    }

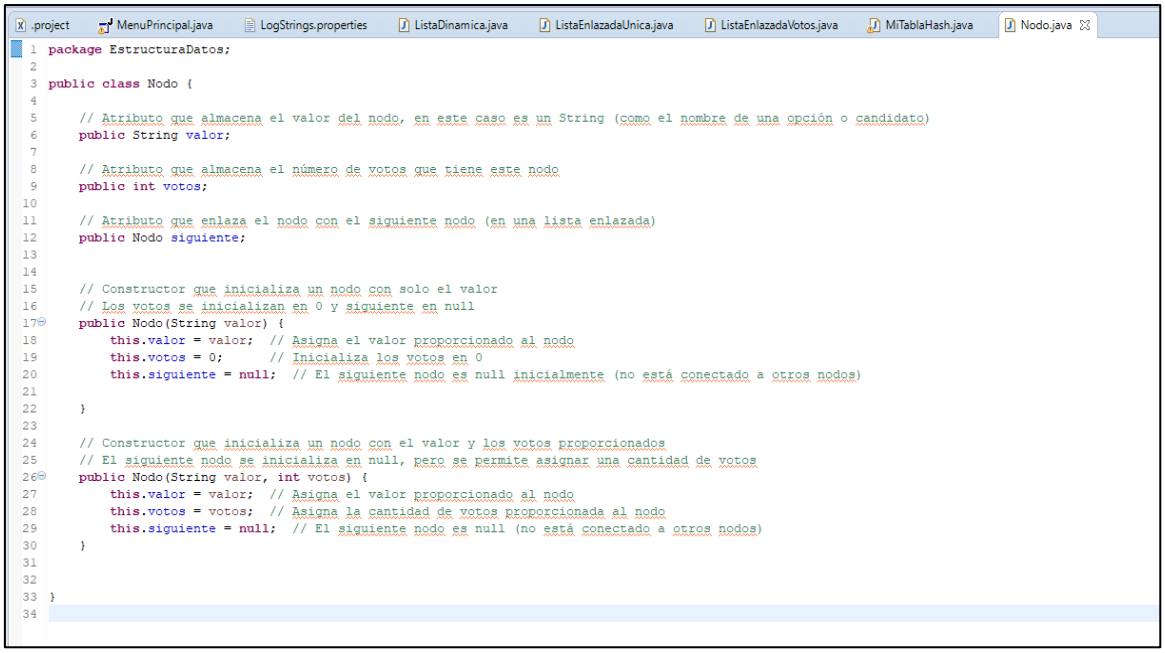
    // Método para insertar una clave y su valor en la tabla hash
    public void insertar(K clave, V valor) {
        int indice = calcularIndice(clave);  // Calculamos el índice usando el hash de la clave
        Nodo<K, V> nuevoNodo = new Nodo<>(clave, valor);  // Creamos un nuevo nodo con la clave y el valor

        // Si no hay ninguna entrada en la posición calculada, insertamos directamente
        if (tabla[indice] == null) {
            tabla[indice] = nuevoNodo;
        } else {
            // Si ya hay una colisión (otra clave en la misma posición), agregamos el nuevo nodo al final de la lista enlazada
            Nodo<K, V> nodoActual = tabla[indice];
            while (nodoActual.siguiente != null) {  // Buscamos el final de la lista enlazada
                nodoActual = nodoActual.siguiente;
            }
            nodoActual.siguiente = nuevoNodo;  // Agregamos el nuevo nodo al final
        }
    }

    // Método para obtener el valor asociado a una clave
    public V obtener(K clave) {
        int indice = calcularIndice(clave);  // Calculamos el índice de la clave
        Nodo<K, V> nodo = tabla[indice];  // Obtenemos el primer nodo en el índice

        // Buscamos en la lista enlazada de ese índice el nodo que tenga la clave
        while (nodo != null) {
```

La clase Nodo representa un elemento de una lista enlazada. Cada nodo contiene un valor (por ejemplo, el nombre de un candidato), un contador de votos y una referencia al siguiente nodo en la lista. Tiene dos constructores: uno que solo asigna el valor al nodo y otro que asigna tanto el valor como los votos iniciales.




```java
package EstructuraDatos;

public class Nodo {

    // Atributo que almacena el valor del nodo, en este caso es un String (como el nombre de una opción o candidato)
    public String valor;

    // Atributo que almacena el número de votos que tiene este nodo
    public int votos;

    // Atributo que enlaza el nodo con el siguiente nodo (en una lista enlazada)
    public Nodo siguiente;

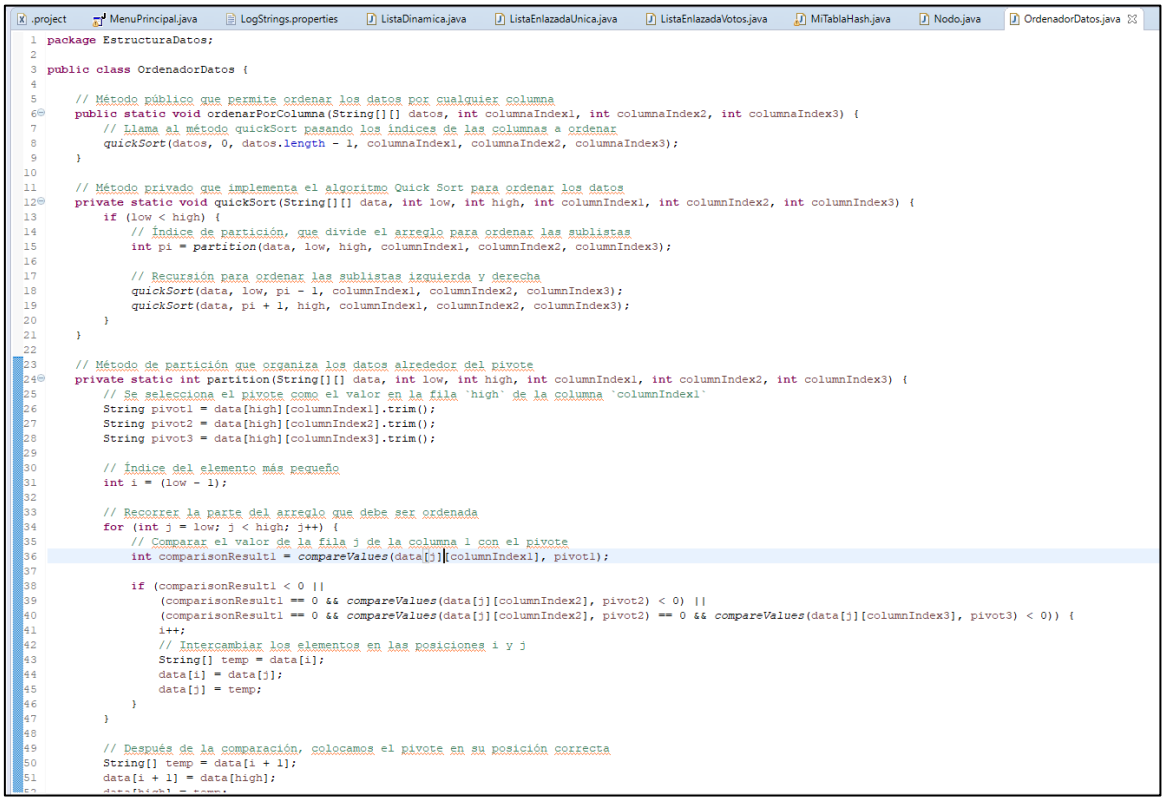

    // Constructor que inicializa un nodo con solo el valor
    // Los votos se inicializan en 0 y siguiente en null
    public Nodo(String valor) {
        this.valor = valor;  // Asigna el valor proporcionado al nodo
        this.votos = 0;      // Inicializa los votos en 0
        this.siguiente = null;  // El siguiente nodo es null inicialmente (no está conectado a otros nodos)

    }

    // Constructor que inicializa un nodo con el valor y los votos proporcionados
    // El siguiente nodo se inicializa en null, pero se permite asignar una cantidad de votos
    public Nodo(String valor, int votos) {
        this.valor = valor;  // Asigna el valor proporcionado al nodo
        this.votos = votos;  // Asigna la cantidad de votos proporcionada al nodo
        this.siguiente = null;  // El siguiente nodo es null (no está conectado a otros nodos)
    }


}
```

La clase OrdenadorDatos implementa un algoritmo de ordenación QuickSort que organiza los datos de una matriz de cadenas en función de varias columnas. El método ordenarPorColumna permite ordenar los datos utilizando los índices de tres columnas. La comparación se realiza primero por la primera columna, luego por la segunda y finalmente por la tercera, si es necesario. El método compareValues permite comparar valores tanto numéricos como cadenas, y el pivote se selecciona de la última fila de la matriz. Esta implementación permite ordenar de manera eficiente y flexible los datos en función de múltiples criterios.




```java
package EstructuraDatos;

public class OrdenadorDatos {

    // Método público que permite ordenar los datos por cualquier columna
    public static void ordenarPorColumna(String[][] datos, int columnaIndex1, int columnaIndex2, int columnaIndex3) {
        // Llama al método quickSort pasando los índices de las columnas a ordenar
        quickSort(datos, 0, datos.length - 1, columnaIndex1, columnaIndex2, columnaIndex3);
    }

    // Método privado que implementa el algoritmo Quick Sort para ordenar los datos
    private static void quickSort(String[][] data, int low, int high, int columnIndex1, int columnIndex2, int columnIndex3) {
        if (low < high) {
            // Índice de partición, que divide el arreglo para ordenar las sublistas
            int pi = partition(data, low, high, columnIndex1, columnIndex2, columnIndex3);

            // Recursión para ordenar las sublistas izquierda y derecha
            quickSort(data, low, pi - 1, columnIndex1, columnIndex2, columnIndex3);
            quickSort(data, pi + 1, high, columnIndex1, columnIndex2, columnIndex3);
        }
    }

    // Método de partición que organiza los datos alrededor del pivote
    private static int partition(String[][] data, int low, int high, int columnIndex1, int columnIndex2, int columnIndex3) {
        // Se selecciona el pivote como el valor en la fila `high` de la columna `columnIndex1`
        String pivot1 = data[high][columnIndex1].trim();
        String pivot2 = data[high][columnIndex2].trim();
        String pivot3 = data[high][columnIndex3].trim();

        // Índice del elemento más pequeño
        int i = (low - 1);

        // Recorrer la parte del arreglo que debe ser ordenada
        for (int j = low; j < high; j++) {
            // Comparar el valor de la fila j de la columna 1 con el pivote
            int comparisonResult1 = compareValues(data[j][columnIndex1], pivot1);

            if (comparisonResult1 < 0 ||
                (comparisonResult1 == 0 && compareValues(data[j][columnIndex2], pivot2) < 0) ||
                (comparisonResult1 == 0 && compareValues(data[j][columnIndex2], pivot2) == 0 && compareValues(data[j][columnIndex3], pivot3) < 0)) {
                i++;
                // Intercambiar los elementos en las posiciones i y j
                String[] temp = data[i];
                data[i] = data[j];
                data[j] = temp;
            }
        }

        // Después de la comparación, colocamos el pivote en su posición correcta
        String[] temp = data[i + 1];
        data[i + 1] = data[high];
```

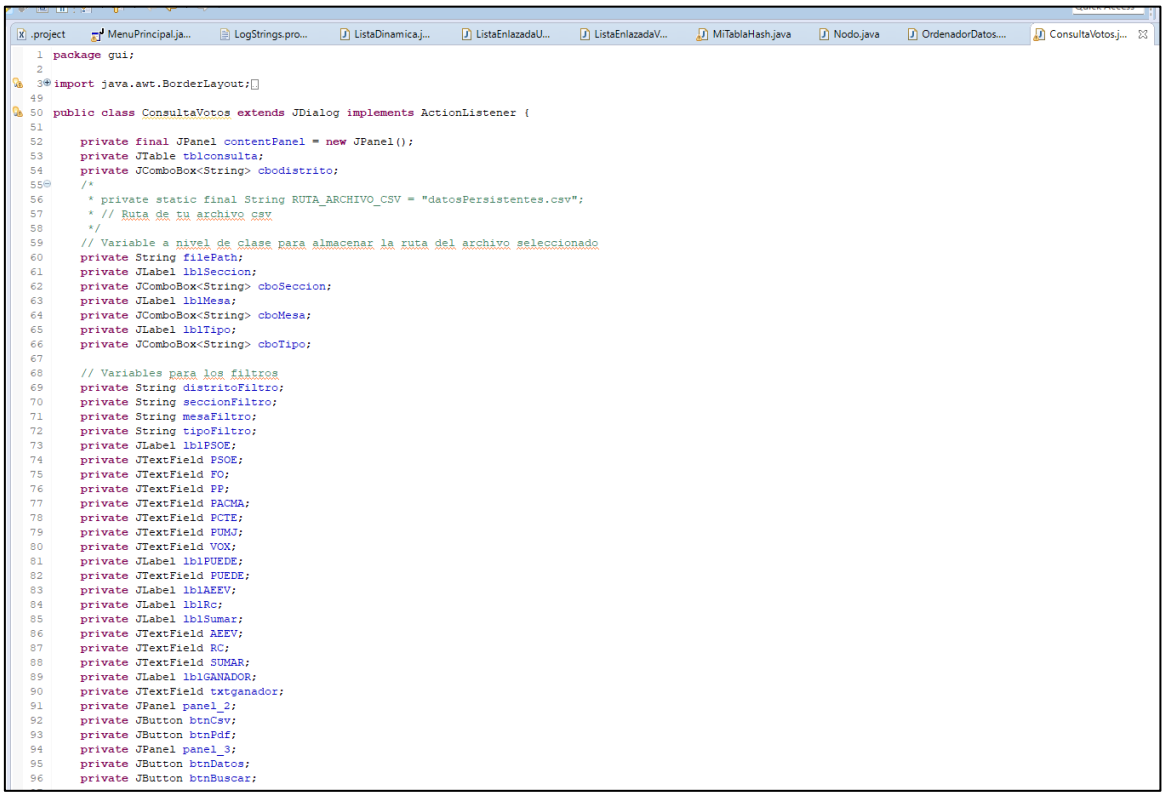
La clase ConsultaVotos se encarga de gestionar y procesar datos relacionados con una consulta de votos. Sus funcionalidades principales incluyen:

Exportar a PDF: Crea un archivo PDF con la información de una tabla de votos.

Exportar a CSV: Permite al usuario guardar los datos de la tabla en un archivo CSV con filtros aplicados.

Importar CSV: Permite al usuario seleccionar e importar datos desde un archivo CSV, cargando estos datos en una tabla y llenando los filtros correspondientes en los combo boxes.

Filtrado de Datos: Los usuarios pueden filtrar los datos mostrados en la tabla según criterios seleccionados en combo boxes (distrito, sección, mesa, tipo de partido).



```java
package gui;

import java.awt.BorderLayout;

public class ConsultaVotos extends JDialog implements ActionListener {

    private final JPanel contentPanel = new JPanel();
    private JTable tblconsulta;
    private JComboBox<String> cbodistrito;
    /*
     * private static final String RUTA_ARCHIVO_CSV = "datosPersistentes.csv";
     * // Ruta de tu archivo csv
     */
    // Variable a nivel de clase para almacenar la ruta del archivo seleccionado
    private String filePath;
    private JLabel lblSeccion;
    private JComboBox<String> cboSeccion;
    private JLabel lblMesa;
    private JComboBox<String> cboMesa;
    private JLabel lblTipo;
    private JComboBox<String> cboTipo;

    // Variables para los filtros
    private String distritoFiltro;
    private String seccionFiltro;
    private String mesaFiltro;
    private String tipoFiltro;
    private JLabel lblPSOE;
    private JTextField PSOE;
    private JTextField FO;
    private JTextField PP;
    private JTextField PACMA;
    private JTextField PCTE;
    private JTextField PUMJ;
    private JTextField VOX;
    private JLabel lblPUEDE;
    private JTextField PUEDE;
    private JLabel lblAEEV;
    private JLabel lblRc;
    private JLabel lblSumar;
    private JTextField AEEV;
    private JTextField RC;
    private JTextField SUMAR;
    private JLabel lblGANADOR;
    private JTextField txtganador;
    private JPanel panel_2;
    private JButton btnCsv;
    private JButton btnPdf;
    private JPanel panel_3;
    private JButton btnDatos;
    private JButton btnBuscar;
```

```java
/**
 * Create the dialog.
 */
public ConsultaVotos() {
    setTitle("Consultas de votaciones");
    setBounds(100, 100, 1238, 452);
    getContentPane().setLayout(new BorderLayout());
    contentPanel.setBorder(new EmptyBorder(5, 5, 5, 5));
    getContentPane().add(contentPanel, BorderLayout.CENTER);
    contentPanel.setLayout(null);

    JScrollPane scrollPane = new JScrollPane();
    scrollPane.setBounds(10, 167, 1086, 236);
    contentPanel.add(scrollPane);

    tblconsulta = new JTable();
    scrollPane.setViewportView(tblconsulta);

    JPanel panel = new JPanel();
    panel.setBorder(new TitledBorder(UIManager.getBorder("TitledBorder.border"), "Formulario de Busquedad",
            TitledBorder.LEADING, TitledBorder.TOP, null, new Color(0, 0, 255)));
    panel.setToolTipText("");
    panel.setBounds(10, 11, 579, 149);
    contentPanel.add(panel);
    panel.setLayout(null);

    cbodistrito = new JComboBox();
    cbodistrito.setBounds(103, 37, 160, 20);
    panel.add(cbodistrito);
    // Escuchar los cambios de selecci\u0000n en el JComboBox
    cbodistrito.addActionListener(this);

    JLabel lblDistrito = new JLabel("Distrito");
    lblDistrito.setBounds(36, 39, 46, 14);
    panel.add(lblDistrito);
    lblDistrito.setFont(new Font("Tahoma", Font.PLAIN, 12));

    cboSeccion = new JComboBox();
    cboSeccion.setBounds(391, 37, 160, 20);
    panel.add(cboSeccion);
    cboSeccion.addActionListener(this);

    lblSeccion = new JLabel("Seccion");
    lblSeccion.setBounds(309, 39, 46, 14);
    panel.add(lblSeccion);
    lblSeccion.setFont(new Font("Tahoma", Font.PLAIN, 12));

    cboTipo = new JComboBox();
    cboTipo.setBounds(391, 82, 160, 20);
    panel.add(cboTipo);
```

```
    public void exportarCSV() {
        // Crear el JFileChooser para permitir al usuario elegir la ubicaci◆n y
        // el nombre del archivo CSV
        JFileChooser fileChooser = new JFileChooser();
        fileChooser.setDialogTitle("Guardar archivo como CSV");

        // Configurar el nombre predeterminado del archivo
        fileChooser.setSelectedFile(new File("reporte_Consulta_votos.csv"));

        // Mostrar el di◆logo para guardar el archivo
        int userSelection = fileChooser.showSaveDialog(this);

        // Si el usuario selecciona "Guardar"
        if (userSelection == JFileChooser.APPROVE_OPTION) {
            // Obtener la ubicaci◆n y el nombre del archivo elegido
            File archivoCSV = fileChooser.getSelectedFile();

            // Si el archivo no termina en ".csv", agregar la extensi◆n
            // autom◆ticamente
            if (!archivoCSV.getName().endsWith(".csv")) {
                archivoCSV = new File(archivoCSV.getAbsolutePath() + ".csv");
            }

            try (FileWriter writer = new FileWriter(archivoCSV)) {
                DefaultTableModel model = (DefaultTableModel) tblconsulta.getModel();

                // Escribir el encabezado del archivo CSV (nombres de las
                // columnas)
                for (int i = 0; i < tblconsulta.getColumnCount(); i++) {
                    writer.append(tblconsulta.getColumnName(i));
                    if (i < tblconsulta.getColumnCount() - 1) {
                        writer.append(";");
                    }
                }
                writer.append("\n");

                // Agregar las filas filtradas a la hoja CSV
                for (int i = 0; i < model.getRowCount(); i++) {
                    // Filtrar las filas en base a los filtros actuales
                    String distrito = (String) model.getValueAt(i, 3);
                    String seccion = (String) model.getValueAt(i, 4);
                    String mesa = (String) model.getValueAt(i, 5);

                    boolean coincide = true;
```

```
                writer.append("\n");

                // Agregar las filas filtradas a la hoja CSV
                for (int i = 0; i < model.getRowCount(); i++) {
                    // Filtrar las filas en base a los filtros actuales
                    String distrito = (String) model.getValueAt(i, 3);
                    String seccion = (String) model.getValueAt(i, 4);
                    String mesa = (String) model.getValueAt(i, 5);

                    boolean coincide = true;

                    // Verificar si los filtros aplicados coinciden
                    if (distritoFiltro != null && !distritoFiltro.equals("Seleccionar...")
                            && !distrito.equals(distritoFiltro)) {
                        coincide = false;
                    }

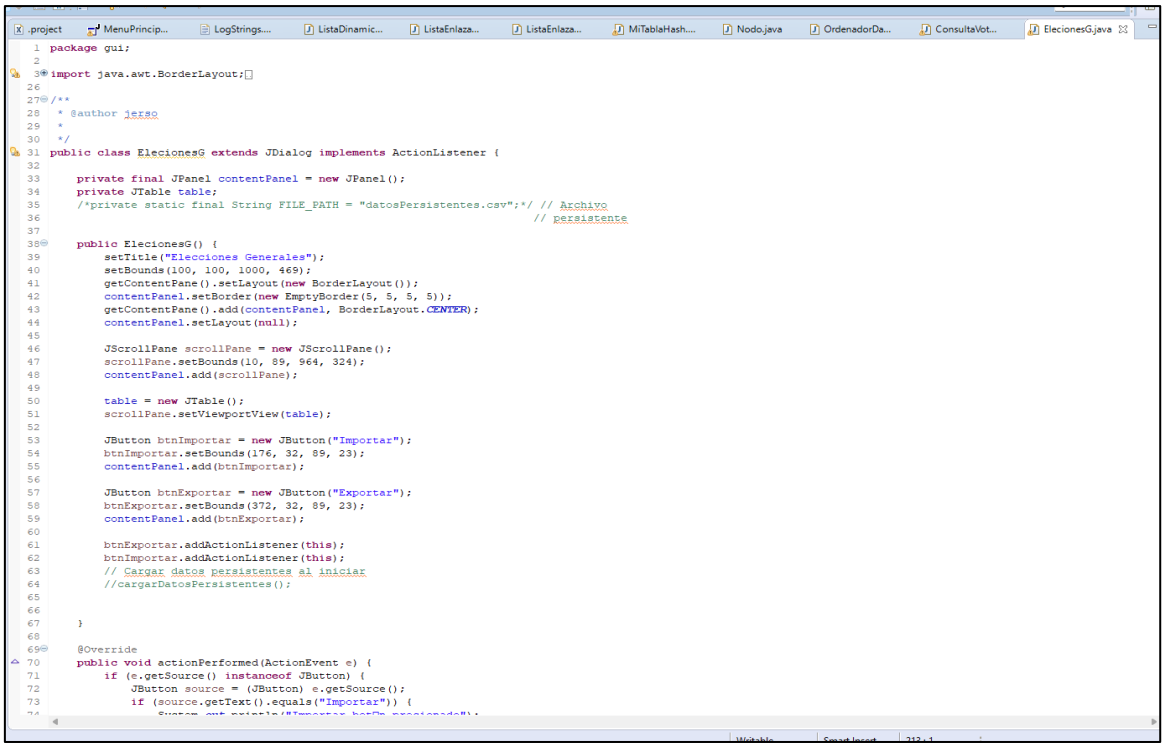
                    if (seccionFiltro != null && !seccionFiltro.equals("Seleccionar...")
                            && !seccion.equals(seccionFiltro)) {
                        coincide = false;
                    }
                    if (mesaFiltro != null && !mesaFiltro.equals("Seleccionar...") && !mesa.equals(mesaFiltro)) {
                        coincide = false;
                    }

                    // Si la fila coincide con los filtros, agregarla al CSV
                    if (coincide) {
                        for (int j = 0; j < model.getColumnCount(); j++) {
                            writer.append((String) model.getValueAt(i, j));
                            if (j < model.getColumnCount() - 1) {
                                writer.append(";");
                            }
                        }
                        writer.append("\n");
                    }
                }

                JOptionPane.showMessageDialog(this,
                        "El archivo CSV se ha generado correctamente en: " + archivoCSV.getAbsolutePath());

            } catch (IOException e) {
                e.printStackTrace();
                JOptionPane.showMessageDialog(this, "Error al generar el archivo CSV.");
            }
        } else {
            // Si el usuario cancela la operaci◆n de guardar
            JOptionPane.showMessageDialog(this, "Exportaci□n cancelada.");
        }
    }
```

La clase ElecionesG es una ventana de tipo JDialog que permite importar y exportar datos de elecciones generales en formato CSV. Cuenta con dos botones, uno para importar archivos CSV y cargar sus datos en una tabla (JTable), y otro para exportar los datos de la tabla a un archivo CSV. Los datos importados se visualizan en una tabla con columnas relacionadas con las elecciones, como año, distrito, partido y número de votos. Además, gestiona la selección de archivos mediante un JFileChooser y muestra mensajes de confirmación o advertencia según el estado de las operaciones.



```java
package gui;

import java.awt.BorderLayout;

/**
 * @author jerso
 *
 */
public class ElecionesG extends JDialog implements ActionListener {

    private final JPanel contentPanel = new JPanel();
    private JTable table;
    /*private static final String FILE_PATH = "datosPersistentes.csv";*/ // Archivo
                                                                         // persistente

    public ElecionesG() {
        setTitle("Elecciones Generales");
        setBounds(100, 100, 1000, 469);
        getContentPane().setLayout(new BorderLayout());
        contentPanel.setBorder(new EmptyBorder(5, 5, 5, 5));
        getContentPane().add(contentPanel, BorderLayout.CENTER);
        contentPanel.setLayout(null);

        JScrollPane scrollPane = new JScrollPane();
        scrollPane.setBounds(10, 89, 964, 324);
        contentPanel.add(scrollPane);

        table = new JTable();
        scrollPane.setViewportView(table);

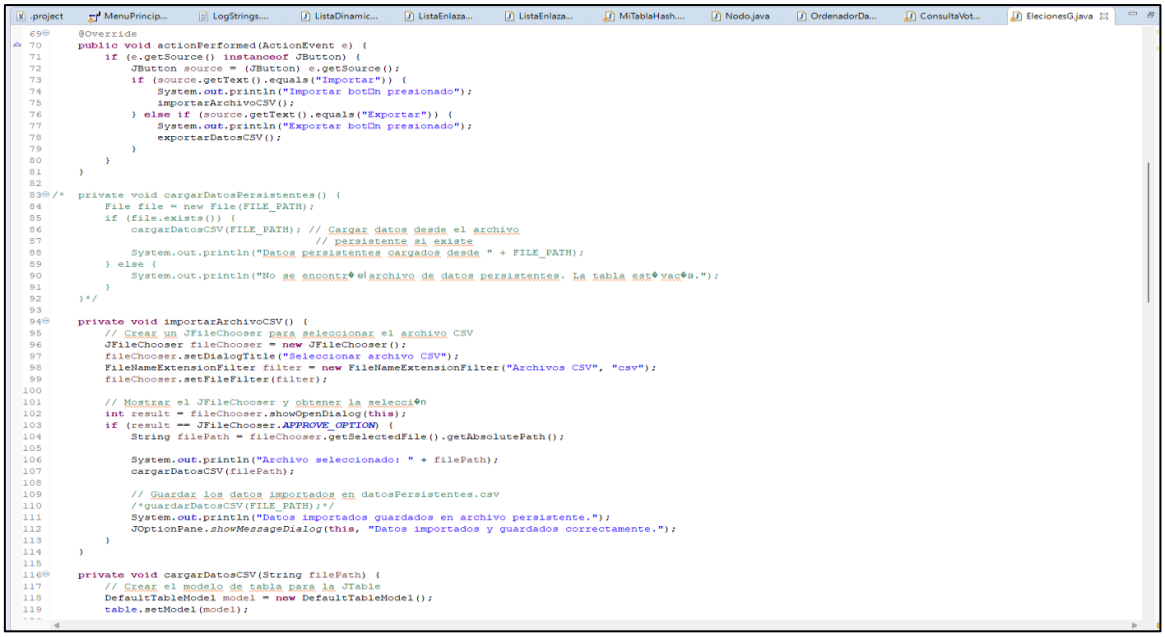
        JButton btnImportar = new JButton("Importar");
        btnImportar.setBounds(176, 32, 89, 23);
        contentPanel.add(btnImportar);

        JButton btnExportar = new JButton("Exportar");
        btnExportar.setBounds(372, 32, 89, 23);
        contentPanel.add(btnExportar);

        btnExportar.addActionListener(this);
        btnImportar.addActionListener(this);
        // Cargar datos persistentes al iniciar
        //cargarDatosPersistentes();


    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() instanceof JButton) {
            JButton source = (JButton) e.getSource();
            if (source.getText().equals("Importar")) {
```



```java
    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() instanceof JButton) {
            JButton source = (JButton) e.getSource();
            if (source.getText().equals("Importar")) {
                System.out.println("Importar botÃn presionado");
                importarArchivoCSV();
            } else if (source.getText().equals("Exportar")) {
                System.out.println("Exportar botÃn presionado");
                exportarDatosCSV();
            }
        }
    }

/*  private void cargarDatosPersistentes() {
        File file = new File(FILE_PATH);
        if (file.exists()) {
            cargarDatosCSV(FILE_PATH); // Cargar datos desde el archivo
                                       // persistente si existe
            System.out.println("Datos persistentes cargados desde " + FILE_PATH);
        } else {
            System.out.println("No se encontrÃ el archivo de datos persistentes. La tabla estÃ vacÃa.");
        }
    }*/

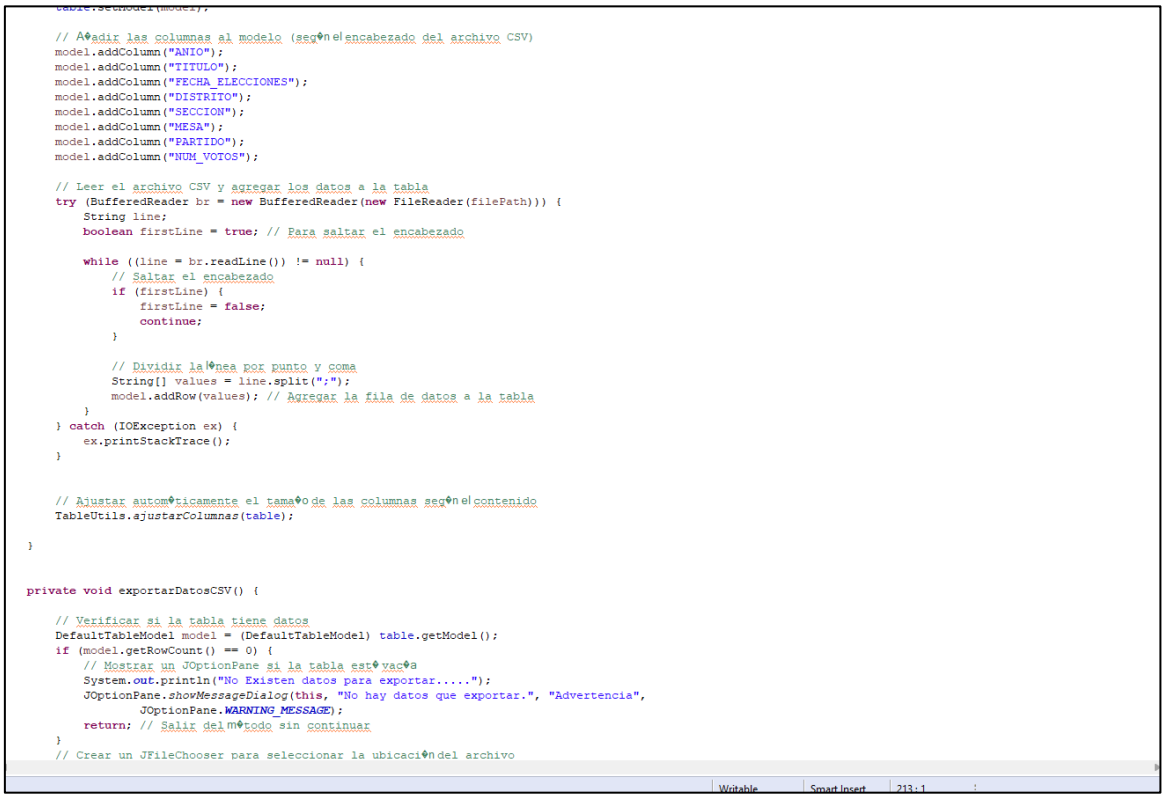
    private void importarArchivoCSV() {
        // Crear un JFileChooser para seleccionar el archivo CSV
        JFileChooser fileChooser = new JFileChooser();
        fileChooser.setDialogTitle("Seleccionar archivo CSV");
        FileNameExtensionFilter filter = new FileNameExtensionFilter("Archivos CSV", "csv");
        fileChooser.setFileFilter(filter);

        // Mostrar el JFileChooser y obtener la selecciÃn
        int result = fileChooser.showOpenDialog(this);
        if (result == JFileChooser.APPROVE_OPTION) {
            String filePath = fileChooser.getSelectedFile().getAbsolutePath();

            System.out.println("Archivo seleccionado: " + filePath);
            cargarDatosCSV(filePath);

            // Guardar los datos importados en datosPersistentes.csv
            /*guardarDatosCSV(FILE_PATH);*/
            System.out.println("Datos importados guardados en archivo persistente.");
            JOptionPane.showMessageDialog(this, "Datos importados y guardados correctamente.");
        }
    }

    private void cargarDatosCSV(String filePath) {
        // Crear el modelo de tabla para la JTable
        DefaultTableModel model = new DefaultTableModel();
        table.setModel(model);
```



```
    table.setModel(model);

    // A◆adir las columnas al modelo (seg◆n el encabezado del archivo CSV)
    model.addColumn("ANIO");
    model.addColumn("TITULO");
    model.addColumn("FECHA_ELECCIONES");
    model.addColumn("DISTRITO");
    model.addColumn("SECCION");
    model.addColumn("MESA");
    model.addColumn("PARTIDO");
    model.addColumn("NUM_VOTOS");

    // Leer el archivo CSV y agregar los datos a la tabla
    try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
        String line;
        boolean firstLine = true; // Para saltar el encabezado

        while ((line = br.readLine()) != null) {
            // Saltar el encabezado
            if (firstLine) {
                firstLine = false;
                continue;
            }

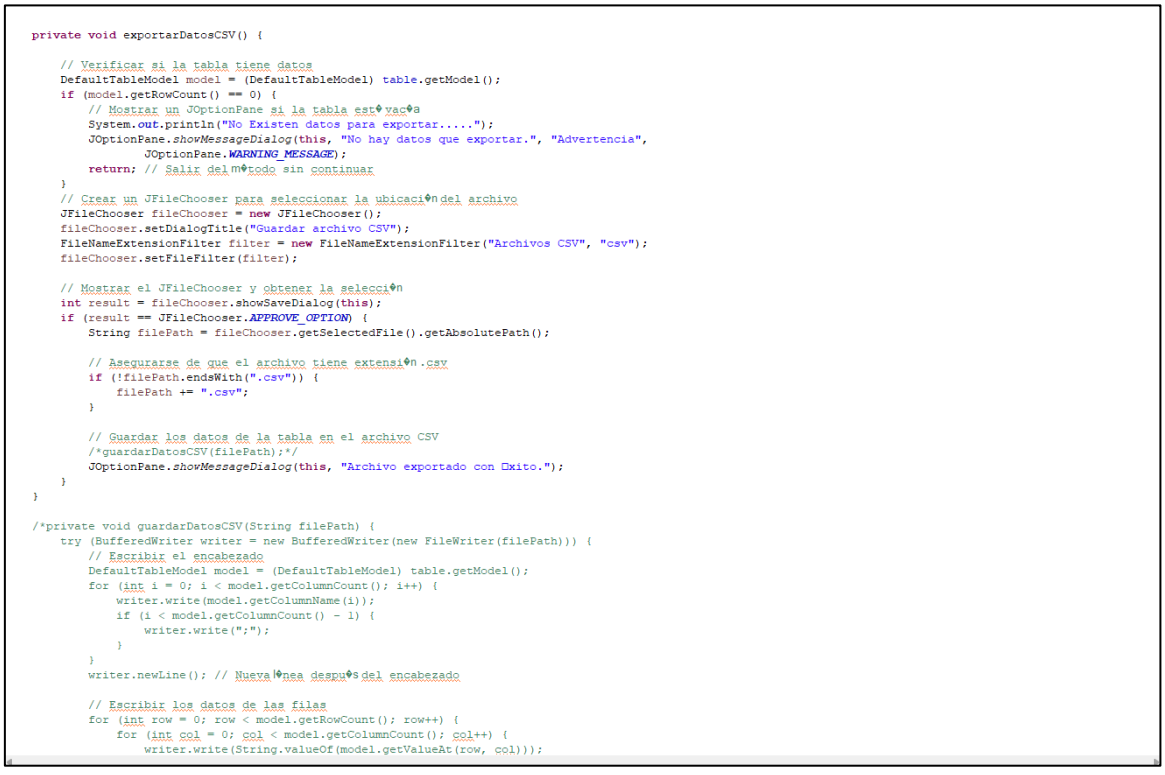
            // Dividir la l◆nea por punto y coma
            String[] values = line.split(";");
            model.addRow(values); // Agregar la fila de datos a la tabla
        }
    } catch (IOException ex) {
        ex.printStackTrace();
    }


    // Ajustar autom◆ticamente el tama◆o de las columnas seg◆n el contenido
    TableUtils.ajustarColumnas(table);

}


private void exportarDatosCSV() {

    // Verificar si la tabla tiene datos
    DefaultTableModel model = (DefaultTableModel) table.getModel();
    if (model.getRowCount() == 0) {
        // Mostrar un JOptionPane si la tabla est◆ vac◆a
        System.out.println("No Existen datos para exportar.....");
        JOptionPane.showMessageDialog(this, "No hay datos que exportar.", "Advertencia",
                JOptionPane.WARNING_MESSAGE);
        return; // Salir del m◆todo sin continuar
    }
    // Crear un JFileChooser para seleccionar la ubicaci◆n del archivo
```



```
private void exportarDatosCSV() {

    // Verificar si la tabla tiene datos
    DefaultTableModel model = (DefaultTableModel) table.getModel();
    if (model.getRowCount() == 0) {
        // Mostrar un JOptionPane si la tabla est◆ vac◆a
        System.out.println("No Existen datos para exportar.....");
        JOptionPane.showMessageDialog(this, "No hay datos que exportar.", "Advertencia",
                JOptionPane.WARNING_MESSAGE);
        return; // Salir del m◆todo sin continuar
    }
    // Crear un JFileChooser para seleccionar la ubicaci◆n del archivo
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setDialogTitle("Guardar archivo CSV");
    FileNameExtensionFilter filter = new FileNameExtensionFilter("Archivos CSV", "csv");
    fileChooser.setFileFilter(filter);

    // Mostrar el JFileChooser y obtener la selecci◆n
    int result = fileChooser.showSaveDialog(this);
    if (result == JFileChooser.APPROVE_OPTION) {
        String filePath = fileChooser.getSelectedFile().getAbsolutePath();

        // Asegurarse de que el archivo tiene extensi◆n .csv
        if (!filePath.endsWith(".csv")) {
            filePath += ".csv";
        }

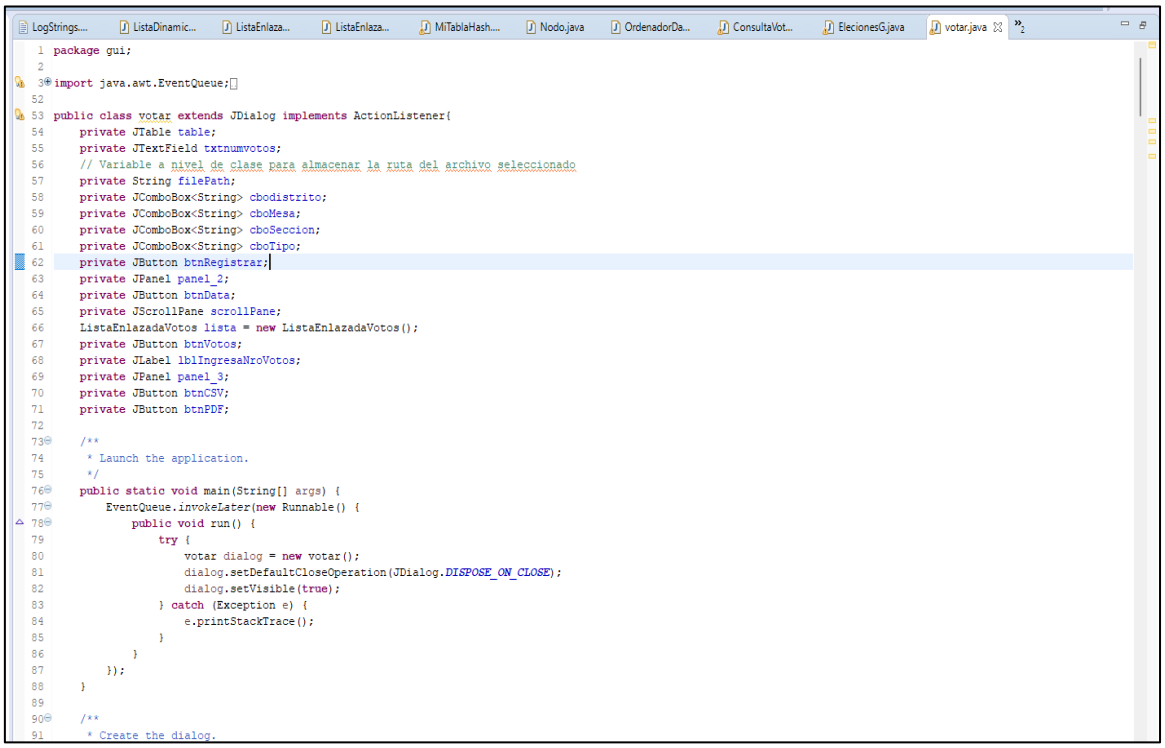
        // Guardar los datos de la tabla en el archivo CSV
        /*guardarDatosCSV(filePath);*/
        JOptionPane.showMessageDialog(this, "Archivo exportado con Exito.");
    }
}

/*private void guardarDatosCSV(String filePath) {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath))) {
        // Escribir el encabezado
        DefaultTableModel model = (DefaultTableModel) table.getModel();
        for (int i = 0; i < model.getColumnCount(); i++) {
            writer.write(model.getColumnName(i));
            if (i < model.getColumnCount() - 1) {
                writer.write(";");
            }
        }
        writer.newLine(); // Nueva l◆nea despu◆s del encabezado

        // Escribir los datos de las filas
        for (int row = 0; row < model.getRowCount(); row++) {
            for (int col = 0; col < model.getColumnCount(); col++) {
                writer.write(String.valueOf(model.getValueAt(row, col)));
```

La clase Votar forma parte de la interfaz gráfica que gestiona la importación, visualización y exportación de datos de votos en un proceso electoral. Utiliza paneles (panel_2 para importar datos y panel_3 para exportar a CSV y PDF) y métodos como inicializar() para desactivar componentes al inicio y ActivarBotones() para habilitarlos después de la importación. Permite

leer datos con leerNumVotos() y obtenerItemSeleccionado(), gestionar la tabla con crearEncabezadoTabla() y ajustarAnchoColumna(), e importar archivos CSV mediante importarArchivoCSV(), que carga los datos en los ComboBoxes. Además, valida y registra los votos con el método RegistrarVoto().



```java
package gui;

import java.awt.EventQueue;

public class votar extends JDialog implements ActionListener{
    private JTable table;
    private JTextField txtnumvotos;
    // Variable a nivel de clase para almacenar la ruta del archivo seleccionado
    private String filePath;
    private JComboBox<String> cbodistrito;
    private JComboBox<String> cboMesa;
    private JComboBox<String> cboSeccion;
    private JComboBox<String> cboTipo;
    private JButton btnRegistrar;
    private JPanel panel_2;
    private JButton btnData;
    private JScrollPane scrollPane;
    ListaEnlazadaVotos lista = new ListaEnlazadaVotos();
    private JButton btnVotos;
    private JLabel lblIngresaNroVotos;
    private JPanel panel_3;
    private JButton btnCSV;
    private JButton btnPDF;

    /**
     * Launch the application.
     */
    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    votar dialog = new votar();
                    dialog.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);
                    dialog.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }

    /**
     * Create the dialog.
```

```java
     */
    public votar() {
        setTitle("Ingrese Votaciones");
        setBounds(100, 100, 999, 404);
        getContentPane().setLayout(null);

        JPanel panel_1 = new JPanel();
        panel_1.setBorder(new TitledBorder(null, "Operaciones", TitledBorder.LEADING, TitledBorder.TOP, null, Color.BLUE));
        panel_1.setBounds(606, 11, 141, 149);
        getContentPane().add(panel_1);
        panel_1.setLayout(null);

        btnRegistrar = new JButton("Registrar");
        btnRegistrar.addActionListener(this);
        btnRegistrar.setBounds(28, 46, 89, 59);
        panel_1.add(btnRegistrar);

        JPanel panel = new JPanel();
        panel.setLayout(null);
        panel.setToolTipText("");
        panel.setBorder(new TitledBorder(UIManager.getBorder("TitledBorder.border"), "Formulario de Registro", TitledBorder.LEADING, TitledBorder.TOP, null, new Colo
        panel.setBounds(17, 11, 579, 149);
        getContentPane().add(panel);

        cbodistrito = new JComboBox();
        cbodistrito.setBounds(103, 36, 160, 20);
        panel.add(cbodistrito);

        JLabel label = new JLabel("Distrito");
        label.setFont(new Font("Tahoma", Font.PLAIN, 12));
        label.setBounds(36, 38, 46, 14);
        panel.add(label);

        cboSeccion = new JComboBox();
        cboSeccion.setBounds(391, 36, 160, 20);
        panel.add(cboSeccion);

        JLabel label_1 = new JLabel("Seccion");
        label_1.setFont(new Font("Tahoma", Font.PLAIN, 12));
        label_1.setBounds(309, 38, 46, 14);
        panel.add(label_1);

        cboTipo = new JComboBox();
        cboTipo.setBounds(391, 81, 160, 20);
        panel.add(cboTipo);

        JLabel label_2 = new JLabel("Tipo");
        label_2.setBounds(309, 84, 46, 14);
        panel.add(label_2);
```

```
     // Placeholder para el campo txtnumvotos
    final String placeholder = "Ingresa Votos";
    txtnumvotos.setText(placeholder);
    txtnumvotos.setForeground(Color.GRAY);  // Color gris para el placeholder

    lblIngresaNroVotos = new JLabel("Ingresa un numero :");
    lblIngresaNroVotos.setBounds(103, 121, 127, 14);
    panel.add(lblIngresaNroVotos);

    // Agregar el FocusListener correctamente
    txtnumvotos.addFocusListener(new FocusListener() {
        @Override
        public void focusGained(FocusEvent e) {
            if (txtnumvotos.getText().equals(placeholder)) {
                txtnumvotos.setText("");  // Eliminar el texto del placeholder al hacer foco
                txtnumvotos.setForeground(Color.BLACK);  // Cambiar color a negro
            }
        }

        @Override
        public void focusLost(FocusEvent e) {
            if (txtnumvotos.getText().isEmpty()) {
                txtnumvotos.setForeground(Color.GRAY);  // Restaurar el color gris
                txtnumvotos.setText(placeholder);  // Restaurar el texto del placeholder
            }
        }
    });

    scrollPane = new JScrollPane();
    scrollPane.setBounds(17, 171, 956, 183);
    getContentPane().add(scrollPane);

    table = new JTable();
    scrollPane.setViewportView(table);

    panel_2 = new JPanel();
    panel_2.setBorder(new TitledBorder(null, "Importar", TitledBorder.LEADING, TitledBorder.TOP, null, Color.BLUE));
    panel_2.setBounds(757, 11, 216, 74);
    getContentPane().add(panel_2);
    panel_2.setLayout(null);

    btnData = new JButton("Data");
    btnData.addActionListener(this);
    btnData.setBounds(66, 11, 89, 23);
    panel_2.add(btnData);

    btnVotos = new JButton("Votos");
```

```
      // Llamar al m◆todo para crear el encabezado de la tabla
     crearEncabezadoTabla();
 // Ajuste de anchos para columnas espec◆ficas

     ajustarAnchoColumna(0, 100); // DISTRITO
     ajustarAnchoColumna(1, 100); // SECCION
     ajustarAnchoColumna(2, 100); // MESA
     ajustarAnchoColumna(3, 80);  // TIPO
     ajustarAnchoColumna(4, 80);  // NUM_VOTOS
     inicializar();
}

// Al iniciar el programa, desactivamos los botones
private void inicializar() {
    btnRegistrar.setEnabled(false);  // Desactivar bot◆n Registrar
    btnVotos.setEnabled(false);  // Desactivar bot◆n Votos
    // Aqu◆ tambi◆n podemos asegurarnos de que los ComboBoxes est◆n vac◆os si no hay datos
    cbodistrito.setEnabled(false);
    cboSeccion.setEnabled(false);
    cboMesa.setEnabled(false);
    cboTipo.setEnabled(false);
    txtnumvotos.setEnabled(false);
}
// Al iniciar el programa, desactivamos los botones
private void ActivarBotones() {
    btnRegistrar.setEnabled(false);  // Desactivar bot◆n Registrar
    btnVotos.setEnabled(true);  // Desactivar bot◆n Votos
    // Aqu◆ tambi◆n podemos asegurarnos de que los ComboBoxes est◆n vac◆os si no hay datos
    cbodistrito.setEnabled(true);
    cboSeccion.setEnabled(true);
    cboMesa.setEnabled(true);
    cboTipo.setEnabled(true);
    txtnumvotos.setEnabled(true);
}

@Override
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == btnPDF) {
        actionPerformedBtnPDF(e);
    }
    if (e.getSource() == btnCSV) {
        actionPerformedBtnCSV(e);
```

```java
@Override
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == btnPDF) {
        actionPerformedBtnPDF(e);
    }
    if (e.getSource() == btnCSV) {
        actionPerformedBtnCSV(e);
    }
    // Verificar si el evento proviene del botón "btnVotos"
    if (e.getSource() == btnVotos) {
        // Si el evento es del botón btnVotos, se llama al metodo actionPerformedBtnVotos
        actionPerformedBtnVotos(e);
    }

    // Verificar si el evento proviene del botón "btnRegistrar"
    if (e.getSource() == btnRegistrar) {
        // Si el evento es del botón btnRegistrar, se llama al metodo actionPerformedBtnRegistrar
        actionPerformedBtnRegistrar(e);
    }

    // Verificar si el evento proviene del botón "btnData"
    if (e.getSource() == btnData) {
        // Si el evento es del botón btnData, se llama al metodo actionPerformedBtnData
        actionPerformedBtnData(e);
    }


}


// Método para leer el número de votos desde un campo de texto y devolverlo como un número entero
int leerNumVotos(){
    // Obtiene el texto del campo de texto 'txtnumvotos', elimina los espacios en blanco al principio y al final,
    // y luego convierte el texto a un valor entero.
    return Integer.parseInt(txtnumvotos.getText().trim());
}

// Método para obtener el item seleccionado de un JComboBox de tipo String
private String obtenerItemSeleccionado(JComboBox<String> comboBox) {
    // Obtiene el objeto seleccionado del comboBox
    Object selectedItem = comboBox.getSelectedItem();

    // Si el objeto seleccionado no es null, lo convierte a String y lo retorna,
    // si es null, retorna una cadena vacía.
    return selectedItem != null ? selectedItem.toString() : "";
}
```

La clase MenuPrincipal es la ventana principal de la aplicación de gestión electoral, que organiza el flujo de operaciones a través de un menú de opciones. Contiene acciones para registrar votos, consultar resultados y generar reportes como votos válidos, nulos, abstenidos y el ganador de las elecciones. Cada opción del menú abre una nueva ventana, utilizando componentes gráficos como JMenuBar, JList y JLabel. Además, emplea un diseño con temas personalizados mediante JTattoo y maneja eventos con métodos como actionPerformed() para ejecutar las acciones correspondientes cuando se seleccionan las opciones del menú.

MenuPrincipal.java

```java
package menu;

import java.awt.BorderLayout;

public class MenuPrincipal extends JFrame implements ActionListener {

    private JPanel contentPane;
    private JMenuBar menuBar;
    /*private static final String LOCK_FILE_PATH = "lockfile.lock";*/ // Ruta del
    private JMenuItem mntmRegistrarVoto;
    private JLabel lblNewLabel;
    private JList<String> list;
    private JList<String> list_2;
    private JMenuItem mntmSalir;
    private JMenuItem mntmConsultarvotos;
    private JMenuItem mntmVotosValidos;
    private JMenuItem mntmVostoNulos;
    private JMenuItem mntmVotosAbstenidos;
    private JMenuItem mntmGanadorDeElecciones;
                                                                    // archivo
                                                                    // de
                                                                    // bloqueo

    /**
     * Launch the application.
     */
    public static void main(String[] args) {
        //aqui pegar el codigo de JTattoo
        //pagina para cambiar de theme Jtattoo https://medium.com/complex-to-simple/java-themes-with-jtattoo-d03d3e72d13c
                try {
                    //aqui puedes poner el nombre de la clase del tema seleccionado en JTattoo
                    UIManager.setLookAndFeel("com.jtattoo.plaf.mcwin.McWinLookAndFeel");

                } catch (ClassNotFoundException ex) {
                    java.util.logging.Logger.getLogger(MenuPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
                } catch (InstantiationException ex) {
```

```java
public MenuPrincipal() {
    setType(Type.UTILITY);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setTitle("Menu Principal");

    addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            System.out.println("AplicaciÓn cerrado correctamente");
            System.exit(0); // Cerrar la aplicaci\u00f3n

        }
    });
    setBounds(100, 100, 454, 391);
    setLocationRelativeTo(null); // Centra la ventana en la pantalla

    menuBar = new JMenuBar();
    setJMenuBar(menuBar);

    JMenu mnAplicacion = new JMenu("Aplicacion");
    menuBar.add(mnAplicacion);

    mntmSalir = new JMenuItem("Salir");
    mntmSalir.addActionListener(this);
    mnAplicacion.add(mntmSalir);

    JMenu mnOperaciones = new JMenu("Operaciones");
    menuBar.add(mnOperaciones);

    mntmRegistrarVoto = new JMenuItem("Registrar Voto");
    mntmRegistrarVoto.addActionListener(this);
    mnOperaciones.add(mntmRegistrarVoto);
```

MenuPrincipal.java

```java
        lblNewLabel.setBounds(10, 11, 422, 102);
        contentPane.add(lblNewLabel);

        list = new JList();
        list.setFont(new Font("Tahoma", Font.BOLD, 11));
        list.setModel(new AbstractListModel() {
            String[] values = new String[] {"Mario Santillana"};
            public int getSize() {
                return values.length;
            }
            public Object getElementAt(int index) {
                return values[index];
            }
        });
        list.setBorder(new TitledBorder(UIManager.getBorder("TitledBorder.border"), "Profesor", TitledBorder.LEADING, TitledBorder.TOP, null, Color.BLUE));
        list.setBounds(10, 163, 422, 50);
        contentPane.add(list);

        JList list_1 = new JList();
        list_1.setFont(new Font("Tahoma", Font.BOLD, 11));
        list_1.setModel(new AbstractListModel() {
            String[] values = new String[] {"- Roberto Agust\u00EDn Mej\u00EDa Collazos", "", "- Miguel \u00C1ngel Vel\u00E1squez Ysuiza", "", "- Manuel \u00C1ngel Pech
            public int getSize() {
                return values.length;
            }
            public Object getElementAt(int index) {
                return values[index];
            }
        });
        list_1.setBorder(new TitledBorder(UIManager.getBorder("TitledBorder.border"), "Integrantes", TitledBorder.LEADING, TitledBorder.TOP, null, Color.BLUE));
        list_1.setBounds(10, 216, 422, 104);
        contentPane.add(list_1);

        list_2 = new JList();
        list_2.setFont(new Font("Tahoma", Font.BOLD, 11));
        list_2.setModel(new AbstractListModel() {
```

MenuPrincipal.java

```java
        list_2 = new JList();
        list_2.setFont(new Font("Tahoma", Font.BOLD, 11));
        list_2.setModel(new AbstractListModel() {
            String[] values = new String[] {"Elecciones Generales Espa\u00F1a"};
            public int getSize() {
                return values.length;
            }
            public Object getElementAt(int index) {
                return values[index];
            }
        });
        list_2.setBorder(new TitledBorder(UIManager.getBorder("TitledBorder.border"), "Proyecto", TitledBorder.LEADING, TitledBorder.TOP, null, new Color(0, 0, 255)));
        list_2.setBounds(10, 124, 422, 39);
        contentPane.add(list_2);
    }

    // Método que se ejecuta cuando se realiza una acción (clic en el menú)
    @Override
    public void actionPerformed(ActionEvent e) {
        // Verifica si el evento proviene de "Ganador de Elecciones" y llama al método correspondiente
        if (e.getSource() == mntmGanadorDeElecciones) {
            actionPerformedMntmGanadorDeElecciones(e);
        }
        // Verifica si el evento proviene de "Votos Abstenidos" y llama al método correspondiente
        if (e.getSource() == mntmVotosAbstenidos) {
            actionPerformedMntmVotosAbstenidos(e);
        }
        // Verifica si el evento proviene de "Votos Nulos" y llama al método correspondiente
        if (e.getSource() == mntmVostoNulos) {
            actionPerformedMntmVostoNulos(e);
        }
        // Verifica si el evento proviene de "Votos Válidos" y llama al método correspondiente
        if (e.getSource() == mntmVotosValidos) {
            actionPerformedMntmVotosValidos(e);
        }
        // Verifica si el evento proviene de "Consultar Votos" y llama al método correspondiente
```

MenuPrincipal.java

```
        }
        // Verifica si el evento proviene de "Votos Válidos" y llama al método correspondiente
        if (e.getSource() == mntmVotosValidos) {
            actionPerformedMntmVotosValidos(e);
        }
        // Verifica si el evento proviene de "Consultar Votos" y llama al método correspondiente
        if (e.getSource() == mntmConsultarvotos) {
            actionPerformedMntmConsultarvotos(e);
        }
        // Verifica si el evento proviene de "Salir" y llama al método correspondiente
        if (e.getSource() == mntmSalir) {
            actionPerformedMntmSalir(e);
        }
        // Verifica si el evento proviene de "Registrar Voto" y llama al método correspondiente
        if (e.getSource() == mntmRegistrarVoto) {
            actionPerformedMntmRegistrarVoto(e);
        }
    }

    // Abre la ventana para registrar un nuevo voto
    protected void actionPerformedMntmRegistrarVoto(ActionEvent e) {
        votar votar = new votar();
        votar.setLocationRelativeTo(this); // Centra la ventana respecto a la ventana principal
        votar.setVisible(true); // Hace visible la ventana de votación
    }

    // Cierra la aplicación
    protected void actionPerformedMntmSalir(ActionEvent e) {
        // Primero muestra un mensaje en la consola indicando que se seleccionó el boton
        System.out.println("Aplicación cerrada correctamente");
        System.exit(0); // Cierra la aplicación con el código de estado 0
    }

    // Abre la ventana para consultar votos
    protected void actionPerformedMntmConsultarvotos(ActionEvent e) {
        // Primero muestra un mensaje en la consola indicando que se seleccionó el boton
```

MenuPrincipal.java

```
    // Abre la ventana para consultar votos
    protected void actionPerformedMntmConsultarvotos(ActionEvent e) {
        // Primero muestra un mensaje en la consola indicando que se seleccionó el boton
        System.out.println("Se seleccionó Consultar Votos");
     // Crea un objeto de tipo ConsultaVotos, que representa la ventana
        ConsultaVotos consvot = new ConsultaVotos();
        consvot.setLocationRelativeTo(this); // Centra la ventana respecto a la ventana principal
        consvot.setVisible(true); // Hace visible la ventana de consulta de votos
    }

    // Abre la ventana que muestra los votos válidos
    protected void actionPerformedMntmVotosValidos(ActionEvent e) {
        // Primero muestra un mensaje en la consola indicando que se seleccionó el boton
        System.out.println("Se seleccionó Votos Válidos");
     // Crea un objeto de tipo VotosValidos, que representa la ventana
        VotosValidos votvali = new VotosValidos();
        votvali.setLocationRelativeTo(this); // Centra la ventana respecto a la ventana principal
        votvali.setVisible(true); // Hace visible la ventana de votos válidos
    }

    // Abre la ventana que muestra los votos nulos
    protected void actionPerformedMntmVostoNulos(ActionEvent e) {
        // Primero muestra un mensaje en la consola indicando que se seleccionó el boton
        System.out.println("Se seleccionó Votos Nulos");
        // Crea un objeto de tipo VotosNulos, que representa la ventana
        VotosNulos votnul = new VotosNulos();
        votnul.setLocationRelativeTo(this); // Centra la ventana respecto a la ventana principal
        votnul.setVisible(true); // Hace visible la ventana de votos nulos
    }

    // Abre la ventana que muestra los votos abstenidos
    protected void actionPerformedMntmVotosAbstenidos(ActionEvent e) {
        // Primero muestra un mensaje en la consola indicando que se seleccionó el boton
        System.out.println("Se seleccionó Votos Abstenidos");
     // Crea un objeto de tipo VotosAbstenidos, que representa la ventana
        VotosAbstenidos votabs = new VotosAbstenidos();
```

MenuPrincipal.java

```
        votvali.setLocationRelativeTo(this); // Centra la ventana respecto a la ventana principal
        votvali.setVisible(true); // Hace visible la ventana de votos válidos
    }

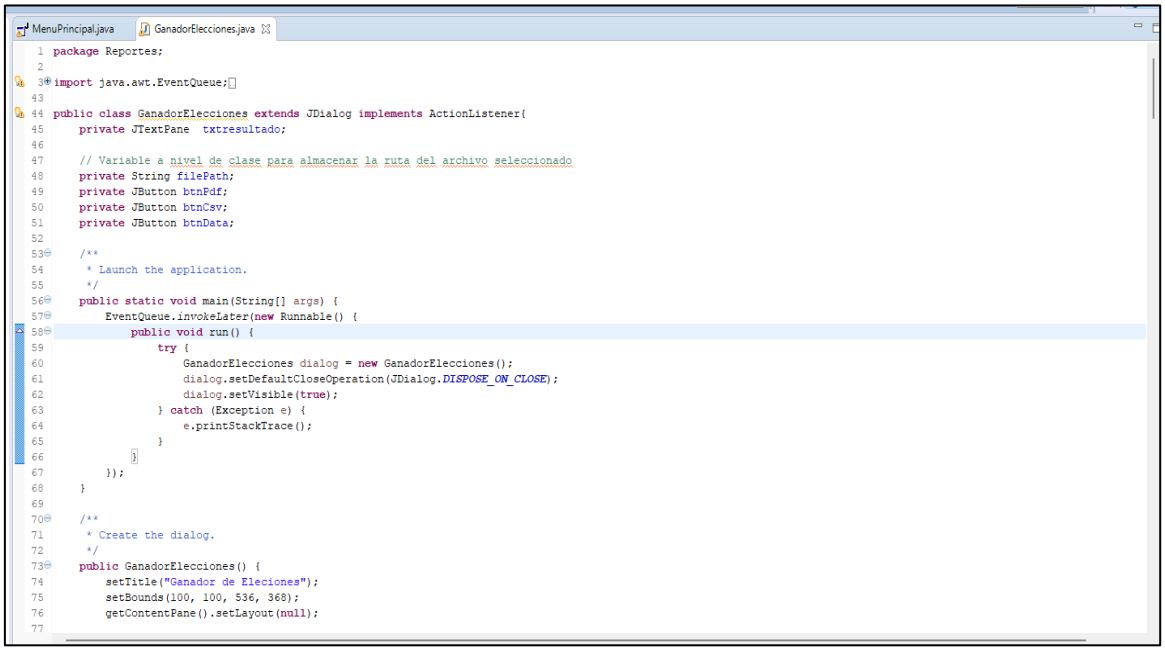
    // Abre la ventana que muestra los votos nulos
    protected void actionPerformedMntmVostoNulos(ActionEvent e) {
        // Primero muestra un mensaje en la consola indicando que se seleccionó el boton
        System.out.println("Se seleccionó Votos Nulos");
        // Crea un objeto de tipo VotosNulos, que representa la ventana
        VotosNulos votnul = new VotosNulos();
        votnul.setLocationRelativeTo(this); // Centra la ventana respecto a la ventana principal
        votnul.setVisible(true); // Hace visible la ventana de votos nulos
    }

    // Abre la ventana que muestra los votos abstenidos
    protected void actionPerformedMntmVotosAbstenidos(ActionEvent e) {
        // Primero muestra un mensaje en la consola indicando que se seleccionó el boton
        System.out.println("Se seleccionó Votos Abstenidos");
     // Crea un objeto de tipo VotosAbstenidos, que representa la ventana
        VotosAbstenidos votabs = new VotosAbstenidos();
        votabs.setLocationRelativeTo(this); // Centra la ventana respecto a la ventana principal
        votabs.setVisible(true); // Hace visible la ventana de votos abstenidos
    }

    // Abre la ventana que muestra el ganador de las elecciones
    protected void actionPerformedMntmGanadorDeElecciones(ActionEvent e) {
        // Primero muestra un mensaje en la consola indicando que se seleccionó el boton
        System.out.println("Se seleccionó Ganador de Elecciones");
        // Crea un objeto de tipo GanadorElecciones, que representa la ventana
        GanadorElecciones ganador = new GanadorElecciones();
        ganador.setLocationRelativeTo(this); // Centra la ventana respecto a la ventana principal
        ganador.setVisible(true); // Hace visible la ventana del ganador de las elecciones
    }

}
```

La clase GanadorElecciones se encarga de gestionar la visualización de los resultados de una elección y la exportación de los datos a diferentes formatos. Esta clase presenta una ventana de diálogo que permite al usuario importar datos desde un archivo CSV, calcular los votos y mostrar el partido ganador. Utiliza botones para generar reportes en PDF o CSV, y maneja los eventos de estos botones a través del método actionPerformed. El método calcularVotosYGanador procesa los datos importados, sumando los votos de cada partido, determinando al ganador, y mostrando los resultados en un JTextPane. Además, permite exportar los resultados a un archivo PDF o CSV mediante los métodos exportarPDF y exportarCSV, utilizando las bibliotecas iText y JFileChooser para crear los archivos y guardarlos en la ubicación elegida por el usuario.



```java
package Reportes;

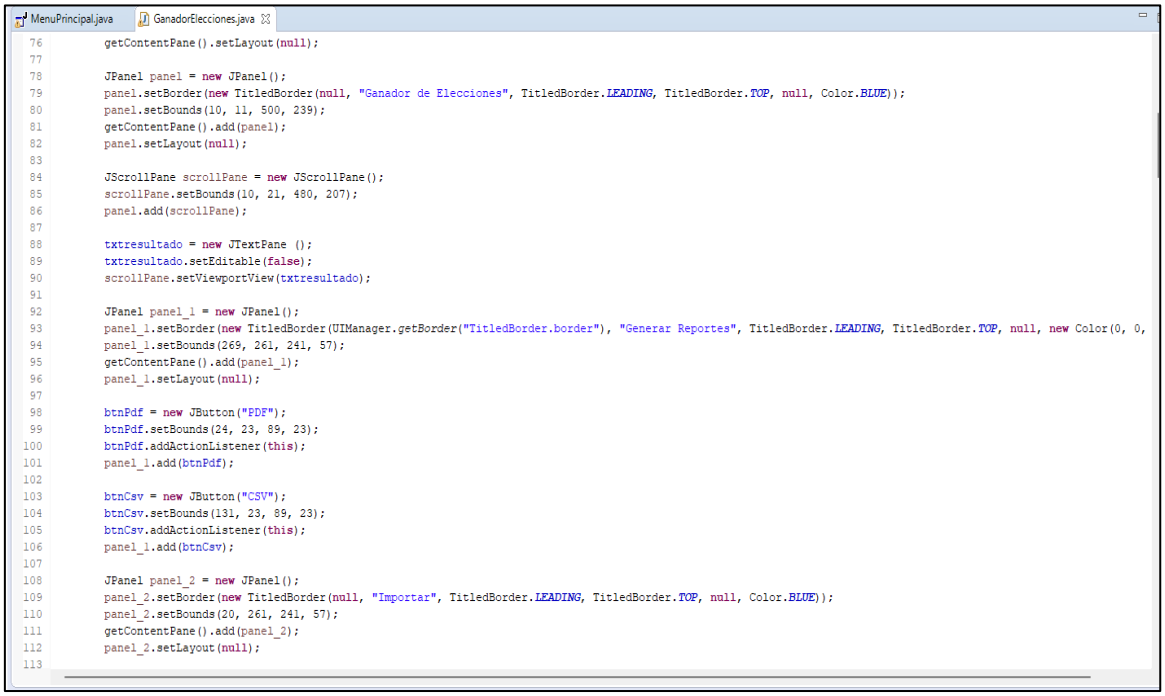
import java.awt.EventQueue;

public class GanadorElecciones extends JDialog implements ActionListener{
    private JTextPane  txtresultado;

    // Variable a nivel de clase para almacenar la ruta del archivo seleccionado
    private String filePath;
    private JButton btnPdf;
    private JButton btnCsv;
    private JButton btnData;

    /**
     * Launch the application.
     */
    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    GanadorElecciones dialog = new GanadorElecciones();
                    dialog.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);
                    dialog.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }

    /**
     * Create the dialog.
     */
    public GanadorElecciones() {
        setTitle("Ganador de Eleciones");
        setBounds(100, 100, 536, 368);
        getContentPane().setLayout(null);
```



```java
        getContentPane().setLayout(null);

        JPanel panel = new JPanel();
        panel.setBorder(new TitledBorder(null, "Ganador de Elecciones", TitledBorder.LEADING, TitledBorder.TOP, null, Color.BLUE));
        panel.setBounds(10, 11, 500, 239);
        getContentPane().add(panel);
        panel.setLayout(null);

        JScrollPane scrollPane = new JScrollPane();
        scrollPane.setBounds(10, 21, 480, 207);
        panel.add(scrollPane);

        txtresultado = new JTextPane ();
        txtresultado.setEditable(false);
        scrollPane.setViewportView(txtresultado);

        JPanel panel_1 = new JPanel();
        panel_1.setBorder(new TitledBorder(UIManager.getBorder("TitledBorder.border"), "Generar Reportes", TitledBorder.LEADING, TitledBorder.TOP, null, new Color(0, 0,
        panel_1.setBounds(269, 261, 241, 57);
        getContentPane().add(panel_1);
        panel_1.setLayout(null);

        btnPdf = new JButton("PDF");
        btnPdf.setBounds(24, 23, 89, 23);
        btnPdf.addActionListener(this);
        panel_1.add(btnPdf);

        btnCsv = new JButton("CSV");
        btnCsv.setBounds(131, 23, 89, 23);
        btnCsv.addActionListener(this);
        panel_1.add(btnCsv);

        JPanel panel_2 = new JPanel();
        panel_2.setBorder(new TitledBorder(null, "Importar", TitledBorder.LEADING, TitledBorder.TOP, null, Color.BLUE));
        panel_2.setBounds(20, 261, 241, 57);
        getContentPane().add(panel_2);
        panel_2.setLayout(null);
```

MenuPrincipal.java | GanadorElecciones.java

```
118
119     }
120
121     @Override
122     public void actionPerformed(ActionEvent e) {
123         // Verifica si el evento proviene del botón "btnPdf"
124         if (e.getSource() == btnPdf) {
125             System.out.println("El botón PDF ha sido presionado."); // Mensaje de verificación
126             exportarPDF(); // Llama a la acción para exportar en PDF
127         }
128         // Verifica si el evento proviene del botón "btnCsv"
129         else if (e.getSource() == btnCsv) {
130             System.out.println("El botón CSV ha sido presionado."); // Mensaje de verificación
131             exportarCSV(); // Llama a la acción para exportar en CSV
132         }
133         // Verifica si el evento proviene del botón "btnData"
134         else if (e.getSource() == btnData) {
135             System.out.println("El botón Data ha sido presionado."); // Mensaje de verificación
136             importarArchivoCSV(); // Llama a la acción para importar desde un archivo CSV
137         }
138     }
139
140
141     private void calcularVotosYGanador() {
142         // Verifica si no se ha importado un archivo CSV (filePath es null o vacío)
143         if (filePath == null || filePath.isEmpty()) {
144             // Muestra un mensaje de advertencia en la consola pidiendo importar un archivo primero
145             System.out.println("Por favor, importe primero un archivo CSV para Calcular votos.");
146             return; // Sale del método si no hay archivo CSV
147         }
148
149         // Crea una instancia de ListaEnlazadaVotos para almacenar votos de cada partido
150         ListaEnlazadaVotos listaVotos = new ListaEnlazadaVotos();
151
152         try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
153             String linea;
154             br.readLine(); // Omite la primera línea, que es el encabezado del archivo CSV
155
```

MenuPrincipal.java | GanadorElecciones.java

```
160
161                 // Verifica que la línea tiene al menos 8 columnas (para evitar errores de índice)
162                 if (datos.length >= 8) {
163                     // Obtiene el nombre del partido (columna 7) y elimina espacios extra
164                     String partido = datos[6].trim();
165
166                     // Verifica que el nombre del partido esté en mayúsculas
167                     if (partido.equals(partido.toUpperCase())) {
168                         // Obtiene el número de votos (columna 8), convirtiéndolo a entero
169                         int votos = Integer.parseInt(datos[7].trim());
170
171                         // Agrega los votos al partido en la lista
172                         listaVotos.agregarVoto(partido, votos);
173                     }
174                 }
175             }
176         } catch (IOException e) {
177             // Si ocurre un error de lectura, muestra el error en la consola
178             e.printStackTrace();
179         }
180
181         // Determina el partido con más votos usando el método obtenerPartidoGanador
182         String partidoGanador = listaVotos.obtenerPartidoGanador();
183
184         // Muestra los votos de todos los partidos en el JTextPane
185         mostrarResultados(listaVotos);
186
187         // Obtiene el documento de texto del JTextPane y prepara un estilo de texto rojo
188         StyledDocument doc = txtresultado.getStyledDocument();
189         Style style = txtresultado.addStyle("RedStyle", null);
190         StyleConstants.setForeground(style, Color.RED); // Establece el color de texto como rojo
191
192         try {
193             // Inserta el nombre del partido ganador en rojo al final del JTextPane
194             doc.insertString(doc.getLength(), "\n\nPartido Ganador: " + partidoGanador, style);
195         } catch (Exception e) {
196             // Si ocurre un error al modificar el documento, lo imprime en la consola
197             e.printStackTrace();
```

MenuPrincipal.java | GanadorElecciones.java

```
190         StyleConstants.setForeground(style, Color.RED); // Establece el color de texto como rojo
191
192         try {
193             // Inserta el nombre del partido ganador en rojo al final del JTextPane
194             doc.insertString(doc.getLength(), "\n\nPartido Ganador: " + partidoGanador, style);
195         } catch (Exception e) {
196             // Si ocurre un error al modificar el documento, lo imprime en la consola
197             e.printStackTrace();
198         }
199     }
200
201
202     private void mostrarResultados(ListaEnlazadaVotos listaVotos) {
203         // Crea un StringBuilder para construir el texto de los resultados de manera eficiente
204         StringBuilder resultado = new StringBuilder();
205
206         // Obtiene el nodo cabeza de la lista enlazada que contiene los votos de los partidos
207         Nodo actual = listaVotos.getCabeza();
208
209         // Recorre cada nodo en la lista enlazada para obtener los datos de cada partido y sus votos
210         while (actual != null) {
211             // Agrega el nombre del partido y la cantidad de votos al StringBuilder
212             resultado.append("Partido: ").append(actual.valor)  // 'valor' representa el nombre del partido
213                      .append("; Votos: ").append(actual.votos)  // 'votos' representa la cantidad de votos del partido
214                      .append("\n");  // Salto de línea para separar cada entrada
215
216             // Avanza al siguiente nodo en la lista enlazada
217             actual = actual.siguiente;
218         }
219
220         // Una vez completado el texto, muestra los resultados en el JTextPane txtresultado
221         txtresultado.setText(resultado.toString());
222     }
223
224
225
226
227
```

MenuPrincipal.java | GanadorElecciones.java

```
226
227
228    private void exportarPDF() {
229        // Crear el JFileChooser para permitir al usuario elegir la ubicación y el nombre del archivo PDF
230        JFileChooser fileChooser = new JFileChooser();
231        fileChooser.setDialogTitle("Guardar archivo como PDF");
232
233        // Configurar el nombre predeterminado del archivo PDF (se puede ajustar según la necesidad)
234        fileChooser.setSelectedFile(new File("reporte_ganador_elecciones.pdf"));
235
236        // Mostrar el cuadro de diálogo para que el usuario seleccione dónde guardar el archivo
237        int userSelection = fileChooser.showSaveDialog(this);
238
239        // Si el usuario aprueba la selección de archivo
240        if (userSelection == JFileChooser.APPROVE_OPTION) {
241            // Obtener el archivo seleccionado
242            File archivo = fileChooser.getSelectedFile();
243
244            // Verificar si el archivo tiene la extensión .pdf; si no, agregarla
245            if (!archivo.getName().endsWith(".pdf")) {
246                archivo = new File(archivo.getAbsolutePath() + ".pdf");
247            }
248
249            try {
250                // Crear el documento PDF
251                Document documento = new Document();
252                PdfWriter.getInstance(documento, new FileOutputStream(archivo));
253                documento.open();
254
255                // Crear una fuente en color rojo para destacar el "Partido Ganador"
256                Font fontRojo = new Font(Font.FontFamily.HELVETICA, 12, Font.NORMAL, BaseColor.RED);
257
258                // Obtener el contenido del JTextPane como texto y dividirlo en líneas
259                String contenido = txtresultado.getText();
260                String[] lineas = contenido.split("\n");
261
262                // Recorrer cada línea del contenido
263                for (String linea : lineas) {
```

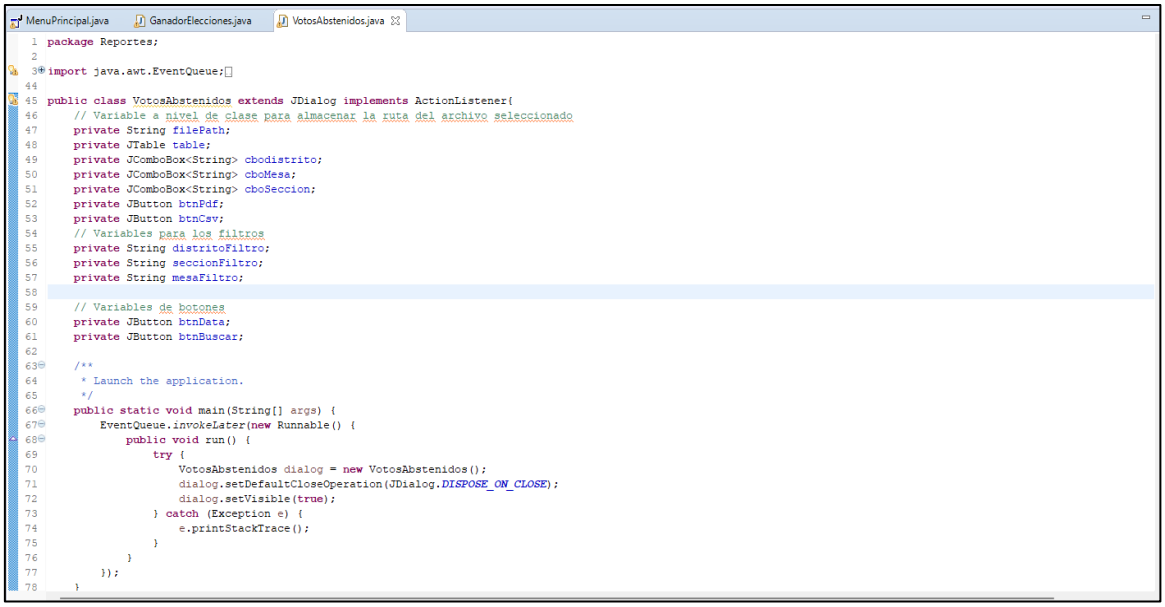
MenuPrincipal.java | GanadorElecciones.java

```
283    }
284
285
286    private void exportarCSV() {
287        // Crear el JFileChooser para permitir al usuario elegir la ubicación y el nombre del archivo CSV
288        JFileChooser fileChooser = new JFileChooser();
289        fileChooser.setDialogTitle("Guardar archivo como CSV");
290
291        // Configurar el nombre predeterminado del archivo (puedes ajustarlo si lo deseas)
292        fileChooser.setSelectedFile(new File("reporte_ganador_elecciones.csv"));
293
294        // Mostrar el diálogo para guardar el archivo
295        int userSelection = fileChooser.showSaveDialog(this);
296
297        // Si el usuario selecciona "Guardar"
298        if (userSelection == JFileChooser.APPROVE_OPTION) {
299            // Obtener la ruta y nombre del archivo seleccionado por el usuario
300            File archivo = fileChooser.getSelectedFile();
301
302            // Si el archivo no tiene la extensión .csv, agregarla automáticamente
303            if (!archivo.getName().endsWith(".csv")) {
304                archivo = new File(archivo.getAbsolutePath() + ".csv");
305            }
306
307            // Intentar escribir en el archivo seleccionado
308            try (FileWriter writer = new FileWriter(archivo)) {
309                // Obtener el contenido que se va a exportar (en este caso, del JTextArea)
310                String contenido = txtresultado.getText();
311
312                // Escribir el contenido en el archivo CSV
313                writer.write(contenido);
314
315                // Mostrar mensaje de éxito al usuario
316                JOptionPane.showMessageDialog(this, "Contenido exportado a CSV exitosamente.");
317            } catch (IOException e) {
318                // Manejar posibles errores durante la escritura del archivo
319                e.printStackTrace();
320                JOptionPane.showMessageDialog(this, "Error al exportar el archivo CSV.");
```

MenuPrincipal.java | GanadorElecciones.java

```
325
326
327    private void importarArchivoCSV() {
328
329        // Crear un JFileChooser para seleccionar el archivo CSV
330        JFileChooser fileChooser = new JFileChooser();
331        fileChooser.setDialogTitle("Seleccionar archivo CSV");  // Título de la ventana del selector de archivos
332        FileNameExtensionFilter filter = new FileNameExtensionFilter("Archivos CSV", "csv");  // Filtro para mostrar solo archivos CSV
333        fileChooser.setFileFilter(filter);  // Aplica el filtro de archivos CSV al JFileChooser
334
335        // Mostrar el JFileChooser y obtener la selección
336        int result = fileChooser.showOpenDialog(this);  // Muestra el diálogo para seleccionar el archivo
337        if (result == JFileChooser.APPROVE_OPTION) {  // Verifica si el usuario ha seleccionado un archivo
338            filePath = fileChooser.getSelectedFile().getAbsolutePath();  // Obtiene la ruta completa del archivo seleccionado
339            System.out.println("Archivo seleccionado: " + filePath);  // Imprime la ruta del archivo seleccionado en la consola
340
341            cargarDatosCSV(filePath);  // Llama al método para cargar los datos del archivo CSV en la tabla
342            calcularVotosYGanador();  // Llama al método para calcular los votos y determinar el ganador
343
344            // Muestra un mensaje indicando que los datos fueron importados y procesados correctamente
345            JOptionPane.showMessageDialog(this, "Datos importados y guardados correctamente.");
346        }
347    }
348
349
350    private void cargarDatosCSV(String filePath) {
351        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {  // Abre el archivo CSV para lectura
352            String linea;  // Variable para almacenar cada línea leída
353            StringBuilder resultado = new StringBuilder();  // StringBuilder para construir el texto que se mostrará
354
355            // Leer cada línea del archivo CSV
356            while ((linea = br.readLine()) != null) {  // Mientras haya líneas en el archivo
357                // Omitir la primera línea si es un encabezado (en este caso, se espera que empiece con "Partido")
358                if (linea.startsWith("Partido")) continue;  // Salta la línea del encabezado
359
360                // Separar los datos de cada línea usando el delimitador ";"
361                String[] datos = linea.split(";");  // Divide la línea en un arreglo de Strings por cada campo separado por ";"
362
```

La clase VotosAbstenidos es responsable de gestionar la visualización de los resultados de los votos abstenidos en las elecciones, permitiendo la importación de datos desde un archivo CSV y aplicando filtros por distrito, sección y mesa. La interfaz de usuario incluye opciones para exportar los resultados a formatos PDF y CSV, utilizando las bibliotecas iText y JFileChooser. El método actionPerformed gestiona las interacciones del usuario, como la selección de filtros y la exportación de los datos. El proceso de filtrado se realiza mediante un conjunto de filtros aplicados a los datos leídos desde el archivo CSV, mostrando solo aquellos que contienen la palabra "abstenciones" en la columna correspondiente. Los resultados son luego presentados en una tabla, y pueden ser exportados a un archivo PDF o CSV según la elección del usuario.



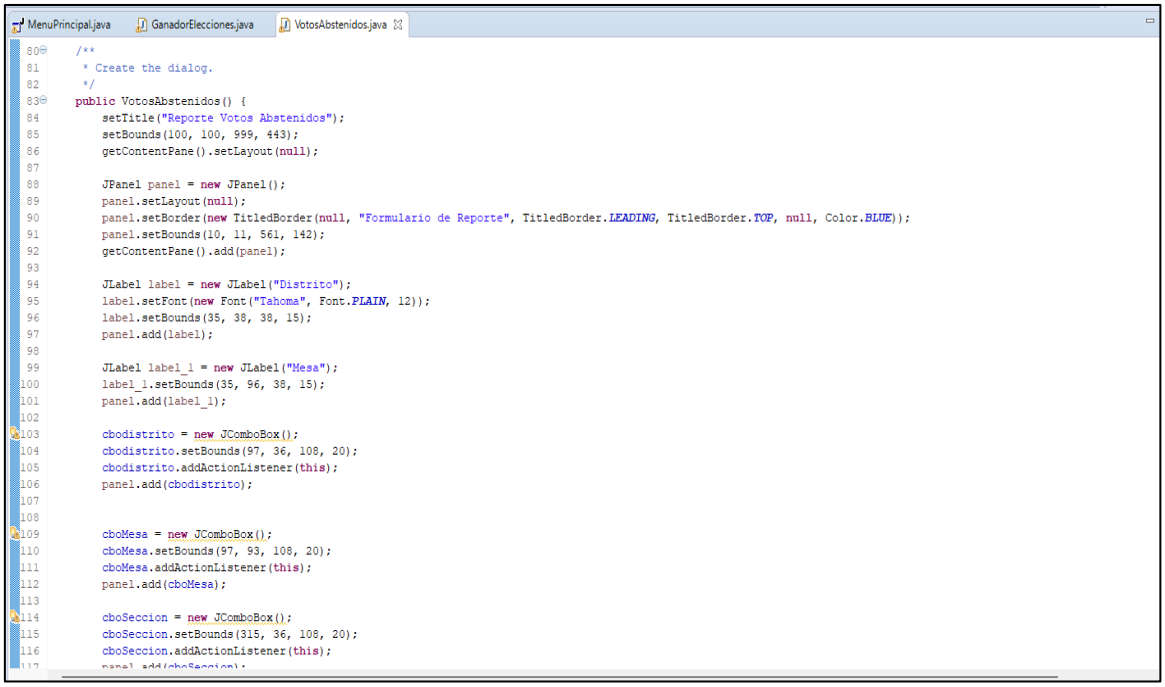

```java
package Reportes;

import java.awt.EventQueue;

public class VotosAbstenidos extends JDialog implements ActionListener{
    // Variable a nivel de clase para almacenar la ruta del archivo seleccionado
    private String filePath;
    private JTable table;
    private JComboBox<String> cbodistrito;
    private JComboBox<String> cboMesa;
    private JComboBox<String> cboSeccion;
    private JButton btnPdf;
    private JButton btnCsv;
    // Variables para los filtros
    private String distritoFiltro;
    private String seccionFiltro;
    private String mesaFiltro;

    // Variables de botones
    private JButton btnData;
    private JButton btnBuscar;

    /**
     * Launch the application.
     */
    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    VotosAbstenidos dialog = new VotosAbstenidos();
                    dialog.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);
                    dialog.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }
```




```java
    /**
     * Create the dialog.
     */
    public VotosAbstenidos() {
        setTitle("Reporte Votos Abstenidos");
        setBounds(100, 100, 999, 443);
        getContentPane().setLayout(null);

        JPanel panel = new JPanel();
        panel.setLayout(null);
        panel.setBorder(new TitledBorder(null, "Formulario de Reporte", TitledBorder.LEADING, TitledBorder.TOP, null, Color.BLUE));
        panel.setBounds(10, 11, 561, 142);
        getContentPane().add(panel);

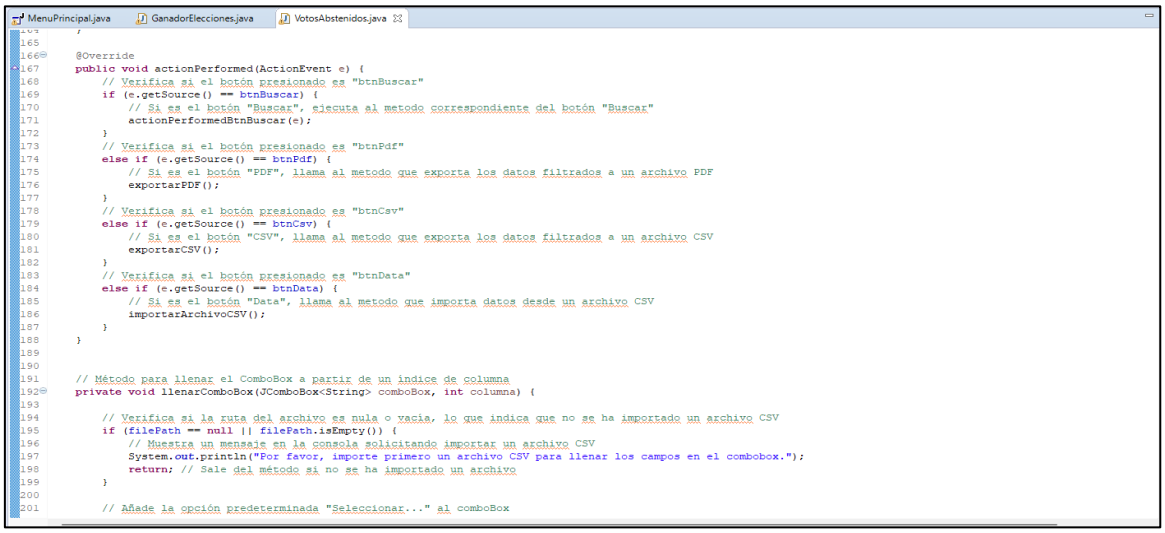
        JLabel label = new JLabel("Distrito");
        label.setFont(new Font("Tahoma", Font.PLAIN, 12));
        label.setBounds(35, 38, 38, 15);
        panel.add(label);

        JLabel label_1 = new JLabel("Mesa");
        label_1.setBounds(35, 96, 38, 15);
        panel.add(label_1);

        cbodistrito = new JComboBox();
        cbodistrito.setBounds(97, 36, 108, 20);
        cbodistrito.addActionListener(this);
        panel.add(cbodistrito);


        cboMesa = new JComboBox();
        cboMesa.setBounds(97, 93, 108, 20);
        cboMesa.addActionListener(this);
        panel.add(cboMesa);

        cboSeccion = new JComboBox();
        cboSeccion.setBounds(315, 36, 108, 20);
        cboSeccion.addActionListener(this);
        panel.add(cboSeccion);
```



```
MenuPrincipal.java  GanadorElecciones.java  VotosAbstenidos.java

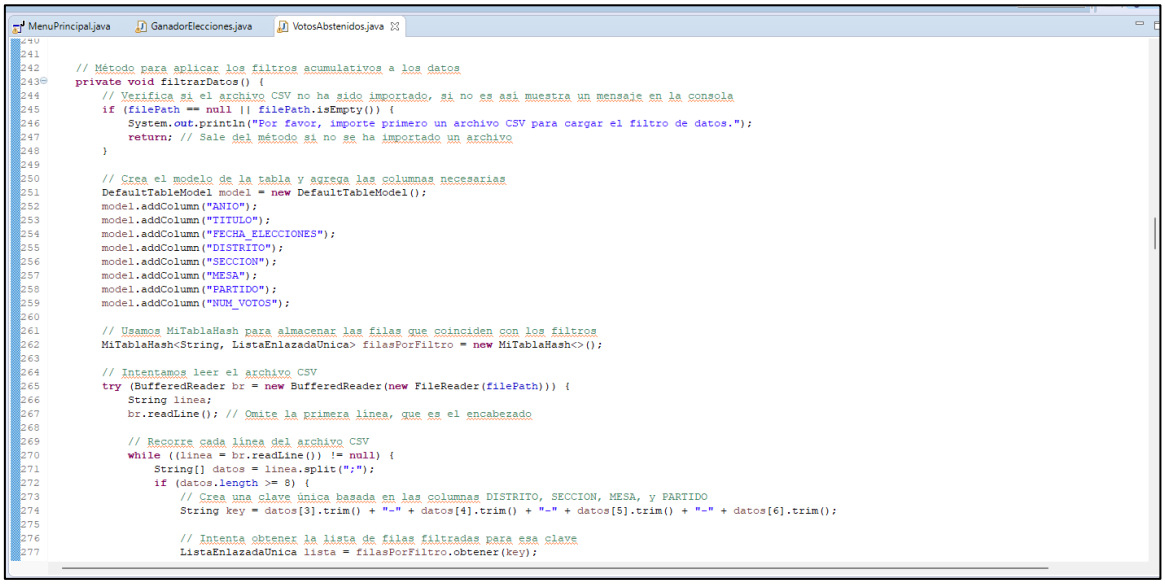
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        // Verifica si el botón presionado es "btnBuscar"
        if (e.getSource() == btnBuscar) {
            // Si es el botón "Buscar", ejecuta al metodo correspondiente del botón "Buscar"
            actionPerformedBtnBuscar(e);
        }
        // Verifica si el botón presionado es "btnPdf"
        else if (e.getSource() == btnPdf) {
            // Si es el botón "PDF", llama al metodo que exporta los datos filtrados a un archivo PDF
            exportarPDF();
        }
        // Verifica si el botón presionado es "btnCsv"
        else if (e.getSource() == btnCsv) {
            // Si es el botón "CSV", llama al metodo que exporta los datos filtrados a un archivo CSV
            exportarCSV();
        }
        // Verifica si el botón presionado es "btnData"
        else if (e.getSource() == btnData) {
            // Si es el botón "Data", llama al metodo que importa datos desde un archivo CSV
            importarArchivoCSV();
        }
    }


    // Método para llenar el ComboBox a partir de un índice de columna
    private void llenarComboBox(JComboBox<String> comboBox, int columna) {

        // Verifica si la ruta del archivo es nula o vacía, lo que indica que no se ha importado un archivo CSV
        if (filePath == null || filePath.isEmpty()) {
            // Muestra un mensaje en la consola solicitando importar un archivo CSV
            System.out.println("Por favor, importe primero un archivo CSV para llenar los campos en el combobox.");
            return; // Sale del método si no se ha importado un archivo
        }

        // Añade la opción predeterminada "Seleccionar..." al comboBox
```



```
MenuPrincipal.java  GanadorElecciones.java  VotosAbstenidos.java


    // Método para aplicar los filtros acumulativos a los datos
    private void filtrarDatos() {
        // Verifica si el archivo CSV no ha sido importado, si no es así muestra un mensaje en la consola
        if (filePath == null || filePath.isEmpty()) {
            System.out.println("Por favor, importe primero un archivo CSV para cargar el filtro de datos.");
            return; // Sale del método si no se ha importado un archivo
        }

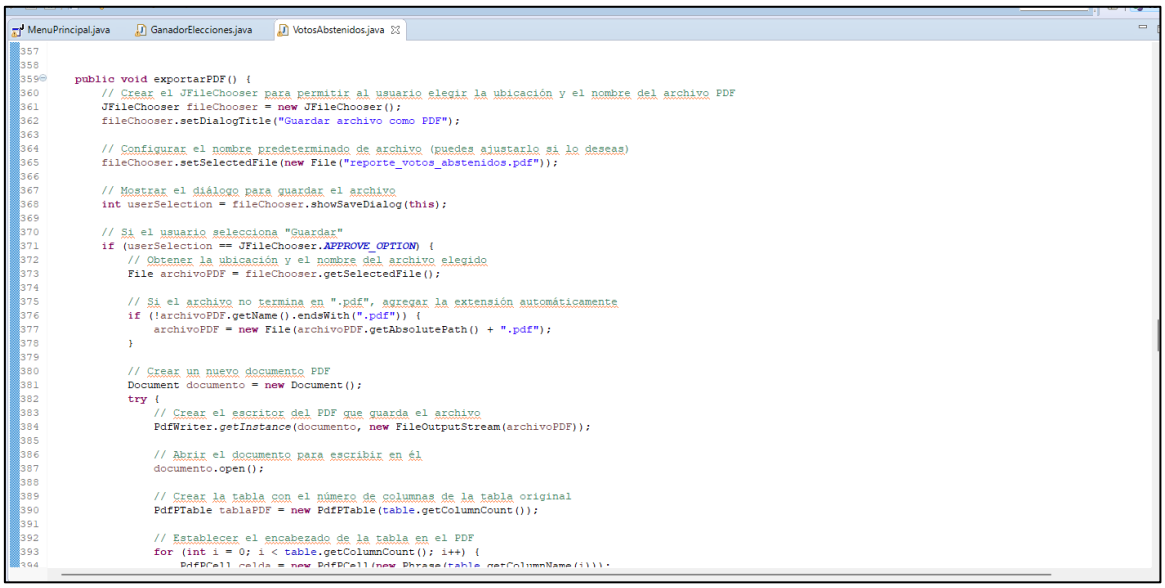
        // Crea el modelo de la tabla y agrega las columnas necesarias
        DefaultTableModel model = new DefaultTableModel();
        model.addColumn("ANIO");
        model.addColumn("TITULO");
        model.addColumn("FECHA_ELECCIONES");
        model.addColumn("DISTRITO");
        model.addColumn("SECCION");
        model.addColumn("MESA");
        model.addColumn("PARTIDO");
        model.addColumn("NUM_VOTOS");

        // Usamos MiTablaHash para almacenar las filas que coinciden con los filtros
        MiTablaHash<String, ListaEnlazadaUnica> filasPorFiltro = new MiTablaHash<>();

        // Intentamos leer el archivo CSV
        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
            String linea;
            br.readLine(); // Omite la primera línea, que es el encabezado

            // Recorre cada línea del archivo CSV
            while ((linea = br.readLine()) != null) {
                String[] datos = linea.split(";");
                if (datos.length >= 8) {
                    // Crea una clave única basada en las columnas DISTRITO, SECCION, MESA, y PARTIDO
                    String key = datos[3].trim() + "-" + datos[4].trim() + "-" + datos[5].trim() + "-" + datos[6].trim();

                    // Intenta obtener la lista de filas filtradas para esa clave
                    ListaEnlazadaUnica lista = filasPorFiltro.obtener(key);
```



```
MenuPrincipal.java  GanadorElecciones.java  VotosAbstenidos.java


    public void exportarPDF() {
        // Crear el JFileChooser para permitir al usuario elegir la ubicación y el nombre del archivo PDF
        JFileChooser fileChooser = new JFileChooser();
        fileChooser.setDialogTitle("Guardar archivo como PDF");

        // Configurar el nombre predeterminado de archivo (puedes ajustarlo si lo deseas)
        fileChooser.setSelectedFile(new File("reporte_votos_abstenidos.pdf"));

        // Mostrar el diálogo para guardar el archivo
        int userSelection = fileChooser.showSaveDialog(this);

        // Si el usuario selecciona "Guardar"
        if (userSelection == JFileChooser.APPROVE_OPTION) {
            // Obtener la ubicación y el nombre del archivo elegido
            File archivoPDF = fileChooser.getSelectedFile();

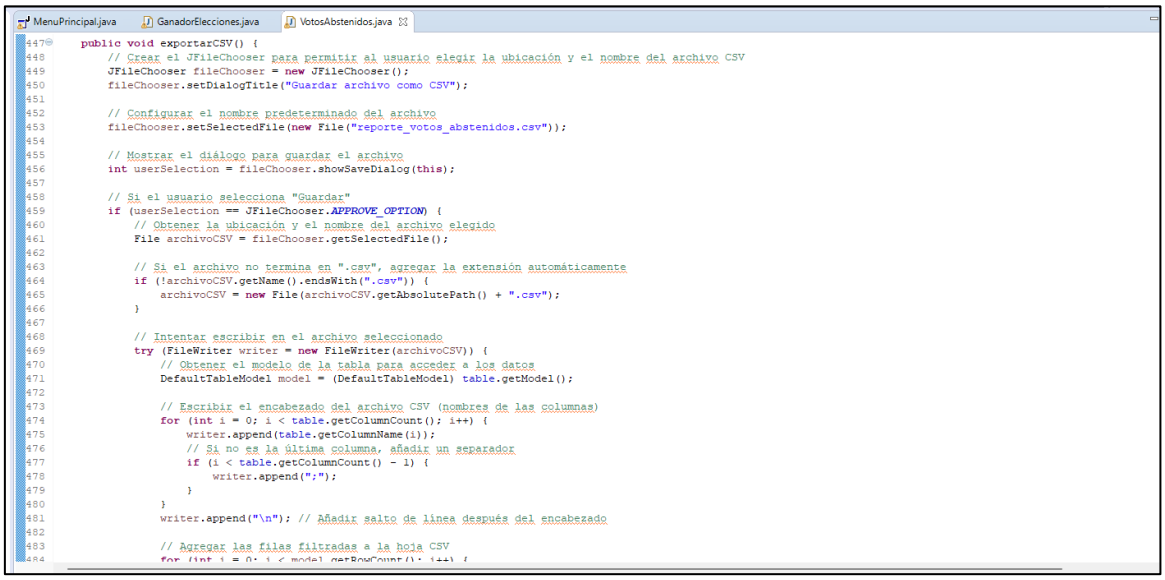
            // Si el archivo no termina en ".pdf", agregar la extensión automáticamente
            if (!archivoPDF.getName().endsWith(".pdf")) {
                archivoPDF = new File(archivoPDF.getAbsolutePath() + ".pdf");
            }

            // Crear un nuevo documento PDF
            Document documento = new Document();
            try {
                // Crear el escritor del PDF que guarda el archivo
                PdfWriter.getInstance(documento, new FileOutputStream(archivoPDF));

                // Abrir el documento para escribir en él
                documento.open();

                // Crear la tabla con el número de columnas de la tabla original
                PdfPTable tablaPDF = new PdfPTable(table.getColumnCount());

                // Establecer el encabezado de la tabla en el PDF
                for (int i = 0; i < table.getColumnCount(); i++) {
```




```
public void exportarCSV() {
    // Crear el JFileChooser para permitir al usuario elegir la ubicación y el nombre del archivo CSV
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setDialogTitle("Guardar archivo como CSV");

    // Configurar el nombre predeterminado del archivo
    fileChooser.setSelectedFile(new File("reporte_votos_abstenidos.csv"));

    // Mostrar el diálogo para guardar el archivo
    int userSelection = fileChooser.showSaveDialog(this);

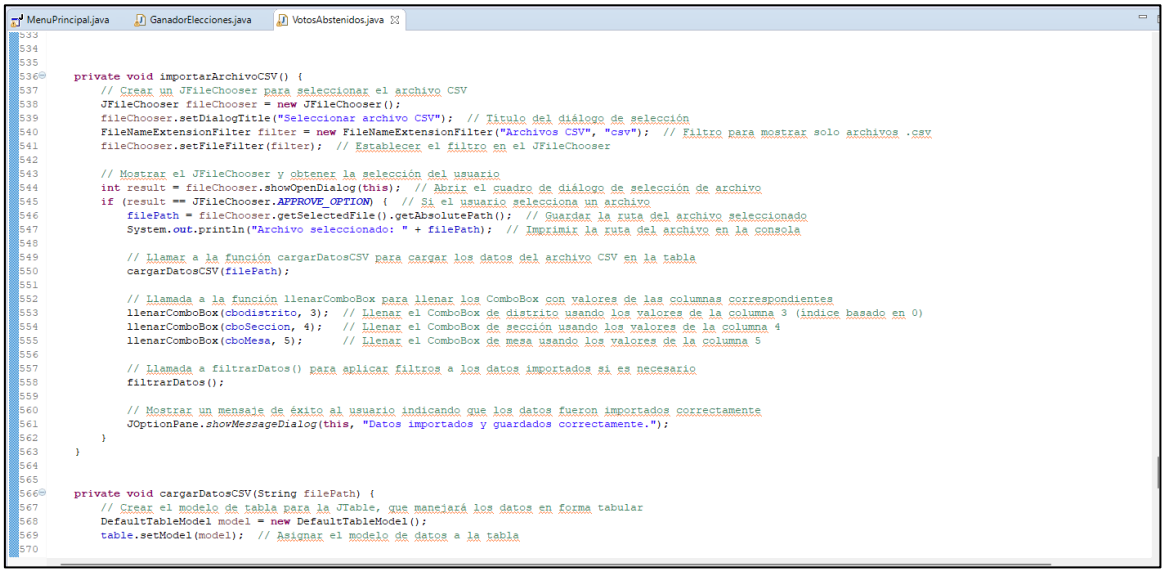
    // Si el usuario selecciona "Guardar"
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        // Obtener la ubicación y el nombre del archivo elegido
        File archivoCSV = fileChooser.getSelectedFile();

        // Si el archivo no termina en ".csv", agregar la extensión automáticamente
        if (!archivoCSV.getName().endsWith(".csv")) {
            archivoCSV = new File(archivoCSV.getAbsolutePath() + ".csv");
        }

        // Intentar escribir en el archivo seleccionado
        try (FileWriter writer = new FileWriter(archivoCSV)) {
            // Obtener el modelo de la tabla para acceder a los datos
            DefaultTableModel model = (DefaultTableModel) table.getModel();

            // Escribir el encabezado del archivo CSV (nombres de las columnas)
            for (int i = 0; i < table.getColumnCount(); i++) {
                writer.append(table.getColumnName(i));
                // Si no es la última columna, añadir un separador
                if (i < table.getColumnCount() - 1) {
                    writer.append(";");
                }
            }
            writer.append("\n"); // Añadir salto de línea después del encabezado

            // Agregar las filas filtradas a la hoja CSV
            for (int i = 0; i < model.getRowCount(); i++) {
```




```
private void importarArchivoCSV() {
    // Crear un JFileChooser para seleccionar el archivo CSV
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setDialogTitle("Seleccionar archivo CSV");  // Título del diálogo de selección
    FileNameExtensionFilter filter = new FileNameExtensionFilter("Archivos CSV", "csv");  // Filtro para mostrar solo archivos .csv
    fileChooser.setFileFilter(filter);  // Establecer el filtro en el JFileChooser

    // Mostrar el JFileChooser y obtener la selección del usuario
    int result = fileChooser.showOpenDialog(this);  // Abrir el cuadro de diálogo de selección de archivo
    if (result == JFileChooser.APPROVE_OPTION) {  // Si el usuario selecciona un archivo
        filePath = fileChooser.getSelectedFile().getAbsolutePath();  // Guardar la ruta del archivo seleccionado
        System.out.println("Archivo seleccionado: " + filePath);  // Imprimir la ruta del archivo en la consola

        // Llamar a la función cargarDatosCSV para cargar los datos del archivo CSV en la tabla
        cargarDatosCSV(filePath);

        // Llamada a la función llenarComboBox para llenar los ComboBox con valores de las columnas correspondientes
        llenarComboBox(cbodistrito, 3);  // Llenar el ComboBox de distrito usando los valores de la columna 3 (índice basado en 0)
        llenarComboBox(cboSeccion, 4);   // Llenar el ComboBox de sección usando los valores de la columna 4
        llenarComboBox(cboMesa, 5);      // Llenar el ComboBox de mesa usando los valores de la columna 5

        // Llamada a filtrarDatos() para aplicar filtros a los datos importados si es necesario
        filtrarDatos();

        // Mostrar un mensaje de éxito al usuario indicando que los datos fueron importados correctamente
        JOptionPane.showMessageDialog(this, "Datos importados y guardados correctamente.");
    }
}

private void cargarDatosCSV(String filePath) {
    // Crear el modelo de tabla para la JTable, que manejará los datos en forma tabular
    DefaultTableModel model = new DefaultTableModel();
    table.setModel(model);  // Asignar el modelo de datos a la tabla
```

La clase VotosNulos gestiona la visualización de los resultados de votos nulos en las elecciones, permitiendo importar datos desde un archivo CSV y aplicar filtros por distrito, sección y mesa. La interfaz de usuario permite exportar los resultados a formatos PDF y CSV utilizando las bibliotecas iText y JFileChooser. El método actionPerformed maneja las interacciones del usuario, como la selección de filtros y la exportación. Los filtros se aplican a los datos leídos desde el archivo CSV, mostrando solo los registros que contienen la palabra "nulos" en la columna correspondiente. Los resultados se muestran en una tabla y pueden ser exportados según la opción seleccionada.

MenuPrincipal.java | GanadorElecciones.java | VotosAbstenidos.java | VotosNulos.java

```java
package Reportes;

import java.awt.EventQueue;

public class VotosNulos extends JDialog implements ActionListener{

    // Variable a nivel de clase para almacenar la ruta del archivo seleccionado
    private String filePath;
    private JTable table;
    private JComboBox<String> cbodistrito;
    private JComboBox<String> cboMesa;
    private JComboBox<String> cboSeccion;
    private JButton btnPdf;
    private JButton btnCsv;

    // Variables para los filtros
    private String distritoFiltro;
    private String seccionFiltro;
    private String mesaFiltro;
    private JButton btnData;
    private JButton btnBuscar;
    /**
     * Launch the application.
     */
    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    VotosNulos dialog = new VotosNulos();
                    dialog.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);
                    dialog.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }

```

MenuPrincipal.java | GanadorElecciones.java | VotosAbstenidos.java | VotosNulos.java

```java
    public VotosNulos() {
        setTitle("Reporte Votos Nulos");
        setBounds(100, 100, 999, 443);
        getContentPane().setLayout(null);

        JPanel panel = new JPanel();
        panel.setLayout(null);
        panel.setBorder(new TitledBorder(null, "Formulario de Reporte", TitledBorder.LEADING, TitledBorder.TOP, null, Color.BLUE));
        panel.setBounds(10, 11, 561, 142);
        getContentPane().add(panel);

        JLabel label = new JLabel("Distrito");
        label.setFont(new Font("Tahoma", Font.PLAIN, 12));
        label.setBounds(35, 38, 38, 15);
        panel.add(label);

        JLabel label_1 = new JLabel("Mesa");
        label_1.setBounds(35, 96, 38, 15);
        panel.add(label_1);

        cbodistrito = new JComboBox();
        cbodistrito.setBounds(97, 36, 108, 20);
        panel.add(cbodistrito);
        cbodistrito.addActionListener(this);

        cboMesa = new JComboBox();
        cboMesa.setBounds(97, 93, 108, 20);
        panel.add(cboMesa);
        cboMesa.addActionListener(this);

        cboSeccion = new JComboBox();
        cboSeccion.setBounds(315, 36, 108, 20);
        panel.add(cboSeccion);
        cboSeccion.addActionListener(this);

        JLabel label_2 = new JLabel("Seccion");
        label_2.setFont(new Font("Tahoma", Font.PLAIN, 12));
```

```java
    public void exportarPDF() {
        // Crear un JFileChooser para permitir al usuario elegir la ubicación y el nombre del archivo PDF
        JFileChooser fileChooser = new JFileChooser();
        fileChooser.setDialogTitle("Guardar archivo como PDF"); // Establecer el título del diálogo

        // Configurar el nombre predeterminado de archivo (puedes ajustarlo si lo deseas)
        fileChooser.setSelectedFile(new File("reporte_votos_nulos.pdf"));

        // Mostrar el diálogo para guardar el archivo
        int userSelection = fileChooser.showSaveDialog(this);

        // Si el usuario selecciona "Guardar"
        if (userSelection == JFileChooser.APPROVE_OPTION) {
            // Obtener la ubicación y el nombre del archivo elegido
            File archivoPDF = fileChooser.getSelectedFile();

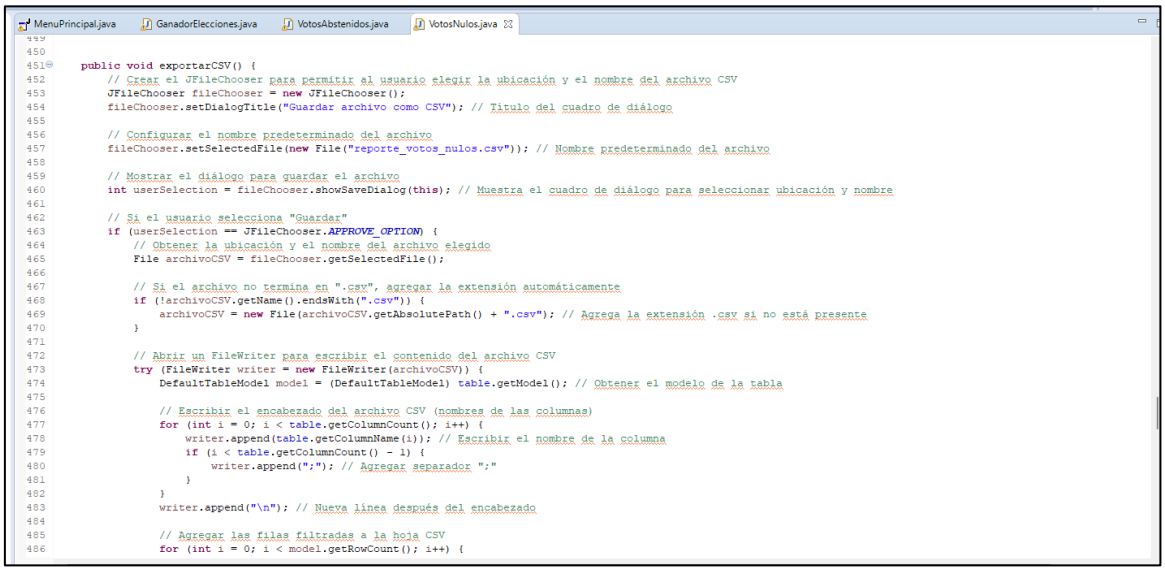
            // Si el archivo no termina en ".pdf", agregar la extensión automáticamente
            if (!archivoPDF.getName().endsWith(".pdf")) {
                archivoPDF = new File(archivoPDF.getAbsolutePath() + ".pdf");
            }

            // Crear un nuevo documento PDF
            Document documento = new Document();
            try {
                // Crear el escritor del PDF que guarda el archivo
                PdfWriter.getInstance(documento, new FileOutputStream(archivoPDF));

                // Abrir el documento para escribir en él
                documento.open();

                // Crear la tabla con el número de columnas de la tabla original
                PdfPTable tablaPDF = new PdfPTable(table.getColumnCount());

                // Establecer el encabezado de la tabla en el PDF
                for (int i = 0; i < table.getColumnCount(); i++) {
```




```
    public void exportarCSV() {
        // Crear el JFileChooser para permitir al usuario elegir la ubicación y el nombre del archivo CSV
        JFileChooser fileChooser = new JFileChooser();
        fileChooser.setDialogTitle("Guardar archivo como CSV"); // Título del cuadro de diálogo

        // Configurar el nombre predeterminado del archivo
        fileChooser.setSelectedFile(new File("reporte_votos_nulos.csv")); // Nombre predeterminado del archivo

        // Mostrar el diálogo para guardar el archivo
        int userSelection = fileChooser.showSaveDialog(this); // Muestra el cuadro de diálogo para seleccionar ubicación y nombre

        // Si el usuario selecciona "Guardar"
        if (userSelection == JFileChooser.APPROVE_OPTION) {
            // Obtener la ubicación y el nombre del archivo elegido
            File archivoCSV = fileChooser.getSelectedFile();

            // Si el archivo no termina en ".csv", agregar la extensión automáticamente
            if (!archivoCSV.getName().endsWith(".csv")) {
                archivoCSV = new File(archivoCSV.getAbsolutePath() + ".csv"); // Agrega la extensión .csv si no está presente
            }

            // Abrir un FileWriter para escribir el contenido del archivo CSV
            try (FileWriter writer = new FileWriter(archivoCSV)) {
                DefaultTableModel model = (DefaultTableModel) table.getModel(); // Obtener el modelo de la tabla

                // Escribir el encabezado del archivo CSV (nombres de las columnas)
                for (int i = 0; i < table.getColumnCount(); i++) {
                    writer.append(table.getColumnName(i)); // Escribir el nombre de la columna
                    if (i < table.getColumnCount() - 1) {
                        writer.append(";"); // Agregar separador ";"
                    }
                }
                writer.append("\n"); // Nueva línea después del encabezado

                // Agregar las filas filtradas a la hoja CSV
                for (int i = 0; i < model.getRowCount(); i++) {
```

```
    private void importarArchivoCSV() {

        // Crear un JFileChooser para permitir al usuario seleccionar el archivo CSV
        JFileChooser fileChooser = new JFileChooser();
        fileChooser.setDialogTitle("Seleccionar archivo CSV"); // Establecer título del diálogo
        FileNameExtensionFilter filter = new FileNameExtensionFilter("Archivos CSV", "csv"); // Filtro para mostrar solo archivos .csv
        fileChooser.setFileFilter(filter); // Aplicar el filtro de archivos CSV al JFileChooser

        // Mostrar el JFileChooser y obtener la selección del usuario
        int result = fileChooser.showOpenDialog(this);

        // Verificar si el usuario seleccionó un archivo (y no canceló)
        if (result == JFileChooser.APPROVE_OPTION) {
            // Obtener la ruta absoluta del archivo seleccionado
            filePath = fileChooser.getSelectedFile().getAbsolutePath();
            System.out.println("Archivo seleccionado: " + filePath); // Imprimir la ruta en consola para depuración

            // Llamada a un método para cargar los datos desde el archivo CSV a la tabla
            cargarDatosCSV(filePath);

            // Llenar los ComboBox automáticamente con los valores únicos de las columnas correspondientes
            // Llenamos el ComboBox del distrito con los datos de la columna 3 (distrito)
            llenarComboBox(cbodistrito, 3); // Columna 3 para distrito
            // Llenamos el ComboBox de sección con los datos de la columna 4 (sección)
            llenarComboBox(cboSeccion, 4);  // Columna 4 para sección
            // Llenamos el ComboBox de mesa con los datos de la columna 5 (mesa)
            llenarComboBox(cboMesa, 5);     // Columna 5 para mesa

            // Aplicar los filtros a los datos en la tabla según las selecciones de los ComboBox
            filtrarDatos();

            // Mostrar un mensaje de éxito al usuario indicando que los datos fueron importados y guardados correctamente
            JOptionPane.showMessageDialog(this, "Datos importados y guardados correctamente.");
        }
    }
```

La clase VotosValidos gestiona la visualización de los resultados de votos válidos en las elecciones, permitiendo importar datos desde un archivo CSV y aplicar filtros por distrito, sección y mesa. La interfaz de usuario ofrece botones para exportar los resultados a formatos PDF y CSV. Los filtros se aplican a los registros importados, mostrando solo aquellos que cumplen con los criterios establecidos (distrito, sección, mesa) y contienen la palabra "válidos" en la columna correspondiente. Los resultados filtrados se visualizan en una tabla, con la opción de exportarlos a los formatos seleccionados.

MenuPrincipal.java | GanadorElecciones.java | VotosAbstenidos.java | VotosNulos.java | VotosValidos.java

```java
/**
 * Create the dialog.
 */
public VotosValidos() {
    setTitle("Reporte Votos Validos");
    setBounds(100, 100, 999, 443);
    getContentPane().setLayout(null);

    JScrollPane scrollPane = new JScrollPane();
    scrollPane.setBounds(10, 157, 963, 236);
    getContentPane().add(scrollPane);

    table = new JTable();
    scrollPane.setViewportView(table);

    JPanel panel = new JPanel();
    panel.setBorder(new TitledBorder(null, "Formulario de Reporte", TitledBorder.LEADING, TitledBorder.TOP, null, Color.BLUE));
    panel.setBounds(10, 11, 561, 142);
    getContentPane().add(panel);
    panel.setLayout(null);

    JLabel label = new JLabel("Distrito");
    label.setBounds(35, 38, 38, 15);
    label.setFont(new Font("Tahoma", Font.PLAIN, 12));
    panel.add(label);

    JLabel label_1 = new JLabel("Mesa");
    label_1.setBounds(35, 96, 38, 15);
    panel.add(label_1);

    cbodistrito = new JComboBox();
    cbodistrito.setBounds(97, 36, 108, 20);
    panel.add(cbodistrito);
    // Escuchar los cambios de selecci\u0000n en el JComboBox
    cbodistrito.addActionListener(this);
```

MenuPrincipal.java | GanadorElecciones.java | VotosAbstenidos.java | VotosNulos.java | VotosValidos.java

```java
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        // Verifica si el evento proviene del botón "Buscar"
        if (e.getSource() == btnBuscar) {
            // Llama a la función que maneja la acción de búsqueda (filtrado de datos)
            actionPerformedBtnBuscar(e);
        }

        // Verifica si el evento proviene del botón "Exportar a PDF"
        else if (e.getSource() == btnPdf) {
            // Llama a la función para exportar los datos filtrados a un archivo PDF
            exportarPDF();
        }

        // Verifica si el evento proviene del botón "Exportar a CSV"
        else if (e.getSource() == btnCsv) {
            // Llama a la función para exportar los datos filtrados a un archivo CSV
            exportarCSV();
        }

        // Verifica si el evento proviene del botón "Importar datos"
        else if (e.getSource() == btnData) {
            // Llama a la función para importar los datos desde un archivo CSV
            importarArchivoCSV();
        }
    }
```

```java

    // Método para llenar los ComboBox a partir de un índice de columna
    private void llenarComboBox(JComboBox<String> comboBox, int columna) {
        // Verificar si el archivo CSV ha sido cargado correctamente
        if (filePath == null || filePath.isEmpty()) {
            // Si no se ha cargado un archivo CSV, muestra un mensaje en la consola
            System.out.println("Por favor, importe primero un archivo CSV para llenar los campos en el combobox.");
            return; // Detiene la ejecución del método si no hay archivo
        }

        // Agregar un item predeterminado "Seleccionar..." al ComboBox
        comboBox.addItem("Seleccionar...");

        // Crear una lista enlazada para almacenar los valores únicos de la columna
        ListaEnlazadaUnica listaUnicos = new ListaEnlazadaUnica();

        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
            // Lee la primera línea del archivo (encabezado) y la omite
            br.readLine();

            String linea;
            // Leer cada línea del archivo CSV
            while ((linea = br.readLine()) != null) {
                // Dividir la línea por el delimitador ";"
                String[] datos = linea.split(";");
                // Verificar si la línea contiene suficientes columnas
                if (datos.length > columna) {
                    // Obtener el valor de la columna especificada y eliminar espacios extra
                    String item = datos[columna].trim();
                    // Agregar el valor a la lista solo si no es un valor repetido
                    listaUnicos.agregar(item);
                }
            }
        } catch (IOException e) {
            // Manejo de errores si ocurre una excepción al leer el archivo
```

MenuPrincipal.java | GanadorElecciones.java | VotosAbstenidos.java | VotosNulos.java | VotosValidos.java

```java
    // Método para aplicar los filtros acumulativos a los datos
    private void filtrarDatos() {
        // Verificar si el archivo CSV ha sido cargado correctamente
        if (filePath == null || filePath.isEmpty()) {
            // Si no se ha cargado un archivo CSV, muestra un mensaje en la consola
            System.out.println("Por favor, importe primero un archivo CSV para cargar el filtro de datos.");
            return; // Detiene la ejecución del método si no hay archivo
        }

        // Crear el modelo de la tabla para representar los datos filtrados
        DefaultTableModel model = new DefaultTableModel();
        model.addColumn("ANIO");
        model.addColumn("TITULO");
        model.addColumn("FECHA_ELECCIONES");
        model.addColumn("DISTRITO");
        model.addColumn("SECCION");
        model.addColumn("MESA");
        model.addColumn("PARTIDO");
        model.addColumn("NUM_VOTOS");

        // Usamos MiTablaHash para almacenar las filas que coinciden con los filtros
        MiTablaHash<String, ListaEnlazadaUnica> filasPorFiltro = new MiTablaHash<>();

        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
            String linea;
            br.readLine(); // Omite el encabezado del archivo CSV
            // Leer cada línea del archivo CSV
            while ((linea = br.readLine()) != null) {
                // Dividir la línea por el delimitador ";"
                String[] datos = linea.split(";");
                // Verificar si la línea contiene al menos 8 columnas
                if (datos.length >= 8) {
                    // Crear una clave única utilizando los valores de las columnas 3, 4, 5 y 6 (DISTRITO, SECCION, MESA, PARTIDO)
                    String key = datos[3].trim() + "-" + datos[4].trim() + "-" + datos[5].trim() + "-" + datos[6].trim();

                    // Crear una lista enlazada para almacenar los datos relacionados con esta clave
                    ListaEnlazadaUnica lista = filasPorFiltro.obtener(key);
                    if (lista == null) {
```

MenuPrincipal.java | GanadorElecciones.java | VotosAbstenidos.java | VotosNulos.java | VotosValidos.java

```java

    public void exportarPDF() {
        // Crear el JFileChooser para permitir al usuario elegir la ubicación y el nombre del archivo PDF
        JFileChooser fileChooser = new JFileChooser();
        fileChooser.setDialogTitle("Guardar archivo como PDF");

        // Configurar el nombre predeterminado de archivo (puedes ajustarlo si lo deseas)
        fileChooser.setSelectedFile(new File("reporte_votos_validos.pdf"));

        // Mostrar el diálogo para guardar el archivo
        int userSelection = fileChooser.showSaveDialog(this);

        // Si el usuario selecciona "Guardar"
        if (userSelection == JFileChooser.APPROVE_OPTION) {
            // Obtener la ubicación y el nombre del archivo elegido
            File archivoPDF = fileChooser.getSelectedFile();

            // Si el archivo no termina en ".pdf", agregar la extensión automáticamente
            if (!archivoPDF.getName().endsWith(".pdf")) {
                archivoPDF = new File(archivoPDF.getAbsolutePath() + ".pdf");
            }

            // Crear un nuevo documento PDF
            Document documento = new Document();
            try {
                // Crear el escritor del PDF que guarda el archivo
                PdfWriter.getInstance(documento, new FileOutputStream(archivoPDF));

                // Abrir el documento para escribir en él
                documento.open();

                // Crear la tabla con el número de columnas de la tabla original
                PdfPTable tablaPDF = new PdfPTable(table.getColumnCount());

                // Establecer el encabezado de la tabla en el PDF
                for (int i = 0; i < table.getColumnCount(); i++) {
                    PdfPCell celda = new PdfPCell(new Phrase(table.getColumnName(i)));
                    tablaPDF.addCell(celda);
```

```java

    public void exportarCSV() {
        // Crear el JFileChooser para permitir al usuario elegir la ubicación y el nombre del archivo CSV
        JFileChooser fileChooser = new JFileChooser();
        fileChooser.setDialogTitle("Guardar archivo como CSV");

        // Configurar el nombre predeterminado del archivo
        fileChooser.setSelectedFile(new File("reporte_votos_validos.csv"));

        // Mostrar el diálogo para guardar el archivo
        int userSelection = fileChooser.showSaveDialog(this);

        // Si el usuario selecciona "Guardar"
        if (userSelection == JFileChooser.APPROVE_OPTION) {
            // Obtener la ubicación y el nombre del archivo elegido
            File archivoCSV = fileChooser.getSelectedFile();

            // Si el archivo no termina en ".csv", agregar la extensión automáticamente
            if (!archivoCSV.getName().endsWith(".csv")) {
                archivoCSV = new File(archivoCSV.getAbsolutePath() + ".csv");
            }

            // Intentar escribir en el archivo CSV usando un FileWriter
            try (FileWriter writer = new FileWriter(archivoCSV)) {
                // Obtener el modelo de la tabla (los datos que están siendo visualizados)
                DefaultTableModel model = (DefaultTableModel) table.getModel();

                // Escribir el encabezado del archivo CSV (nombres de las columnas de la tabla)
                for (int i = 0; i < table.getColumnCount(); i++) {
                    writer.append(table.getColumnName(i));  // Escribe el nombre de la columna
                    if (i < table.getColumnCount() - 1) {  // Si no es la última columna
                        writer.append(";");  // Separa las columnas con un punto y coma
                    }
                }
                writer.append("\n");  // Salto de línea después del encabezado

                // Agregar las filas filtradas a la hoja CSV
```

```java
private void importarArchivoCSV() {

    // Crear un JFileChooser para seleccionar el archivo CSV
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setDialogTitle("Seleccionar archivo CSV");

    // Establecer un filtro para que solo se muestren archivos CSV
    FileNameExtensionFilter filter = new FileNameExtensionFilter("Archivos CSV", "csv");
    fileChooser.setFileFilter(filter);

    // Mostrar el JFileChooser y obtener la selección del archivo
    int result = fileChooser.showOpenDialog(this);

    // Si el usuario selecciona un archivo
    if (result == JFileChooser.APPROVE_OPTION) {

        // Obtener la ruta absoluta del archivo seleccionado
        filePath = fileChooser.getSelectedFile().getAbsolutePath();  // Guardar la ruta seleccionada
        System.out.println("Archivo seleccionado: " + filePath);

        // Llamar al método para cargar los datos del archivo CSV en la tabla
        cargarDatosCSV(filePath);

        // Llamadas para llenar los ComboBox automáticamente después de importar
        // Llenar ComboBox para distrito, columna 3 en la tabla
        llenarComboBox(cbodistrito, 3);

        // Llenar ComboBox para sección, columna 4 en la tabla
        llenarComboBox(cboSeccion, 4);

        // Llenar ComboBox para mesa, columna 5 en la tabla
        llenarComboBox(cboMesa, 5);

        // Aplicar filtros a los datos cargados
        filtrarDatos();
```

```java
private void cargarDatosCSV(String filePath) {
    // Crear el modelo de tabla para la JTable
    DefaultTableModel model = new DefaultTableModel();
    table.setModel(model);  // Establecer el modelo de la tabla como el que acabamos de crear

    // Añadir las columnas al modelo de tabla (según el encabezado del archivo CSV)
    model.addColumn("ANIO");
    model.addColumn("TITULO");
    model.addColumn("FECHA_ELECCIONES");
    model.addColumn("DISTRITO");
    model.addColumn("SECCION");
    model.addColumn("MESA");
    model.addColumn("PARTIDO");
    model.addColumn("NUM_VOTOS");

    // Leer el archivo CSV y agregar los datos a la tabla
    try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
        String line;
        boolean firstLine = true;  // Flag para saltarse el encabezado del archivo CSV

        // Leer cada línea del archivo CSV
        while ((line = br.readLine()) != null) {
            // Saltar el encabezado (primera línea)
            if (firstLine) {
                firstLine = false;  // Establecer el flag a false para no saltar las siguientes líneas
                continue;
            }

            // Dividir la línea de texto por el separador de columnas (punto y coma ";")
            String[] values = line.split(";");

            // Agregar la fila de datos a la tabla
            model.addRow(values);
        }
    } catch (IOException ex) {
        // En caso de error al leer el archivo, se captura y se imprime el error
```

La clase TableUtils contiene un método estático ajustarColumnas que ajusta el ancho de las columnas de una tabla JTable en función del contenido de las celdas. El método recorre cada columna y fila de la tabla, calculando el ancho máximo necesario para cada columna según la longitud del texto de las celdas. Si una celda contiene texto, el ancho de la columna se actualiza para garantizar que todo el contenido sea visible. Finalmente, se ajusta el ancho de la columna multiplicando el valor calculado por un factor de 10 para mejorar la visibilidad del texto.

MenuPrincipal.java | GanadorElecciones.java | VotosAbstenidos.java | VotosNulos.java | VotosValidos.java | TableUtils.java

```java
package Util;

import javax.swing.JTable;

public class TableUtils {

    // Método estático para ajustar el ancho de las columnas de una JTable
    public static void ajustarColumnas(JTable table) {
        // Recorre cada columna de la tabla
        for (int column = 0; column < table.getColumnCount(); column++) {
            int width = 0; // Inicializa el ancho máximo para la columna actual

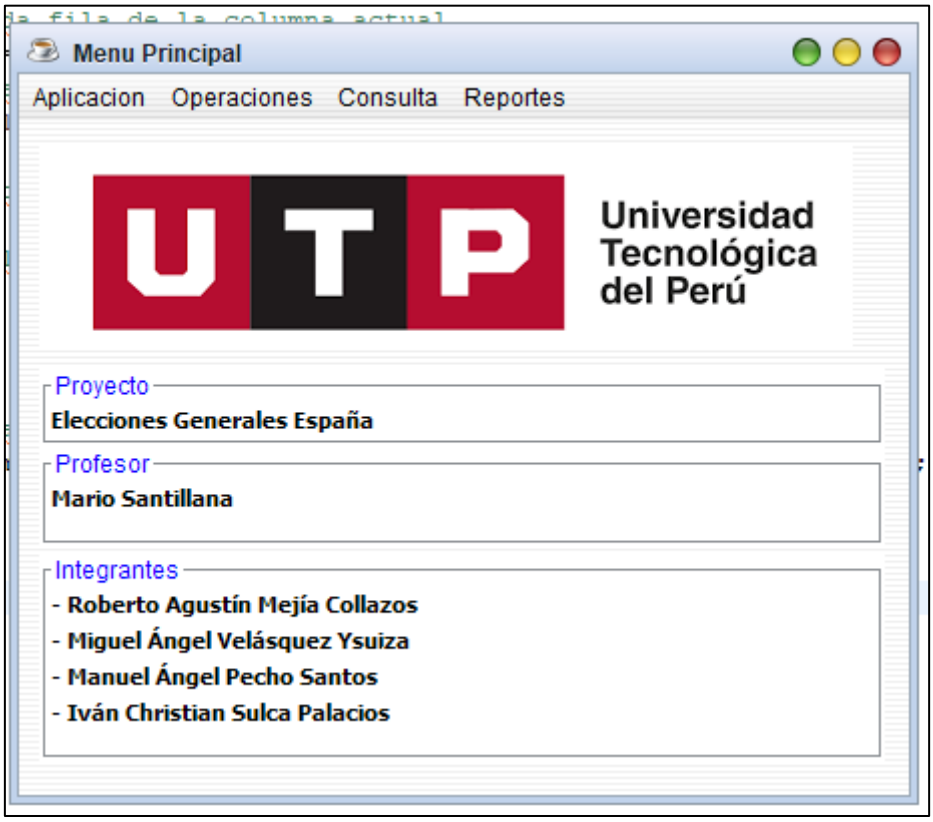
            // Recorre cada fila de la columna actual
            for (int row = 0; row < table.getRowCount(); row++) {
                // Obtiene el valor en la celda (row, column)
                Object value = table.getValueAt(row, column);

                // Si la celda no está vacía, actualiza el ancho máximo
                if (value != null) {
                    // Calcula el ancho según la longitud del texto de la celda
                    width = Math.max(width, value.toString().length());
                }
            }

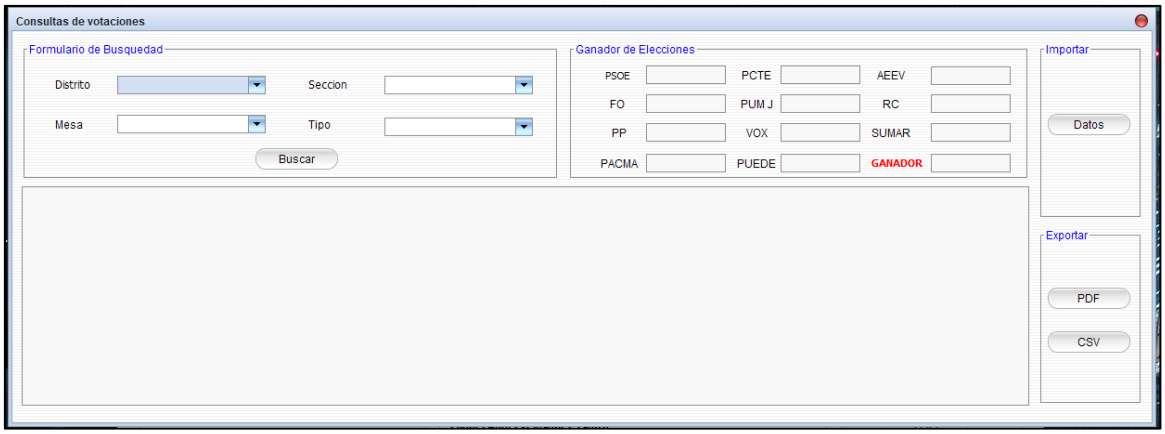
            // Escala el ancho de la columna para mejorar la visibilidad del texto
            table.getColumnModel().getColumn(column).setPreferredWidth(width * 10);
        }
    }
}
```

Pantallas de ejecución

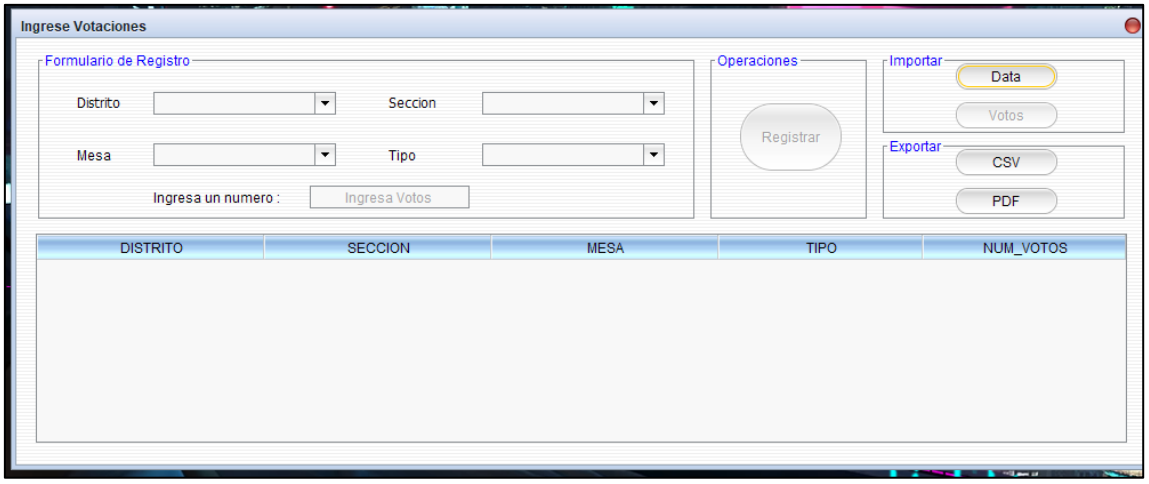
Menu Principal



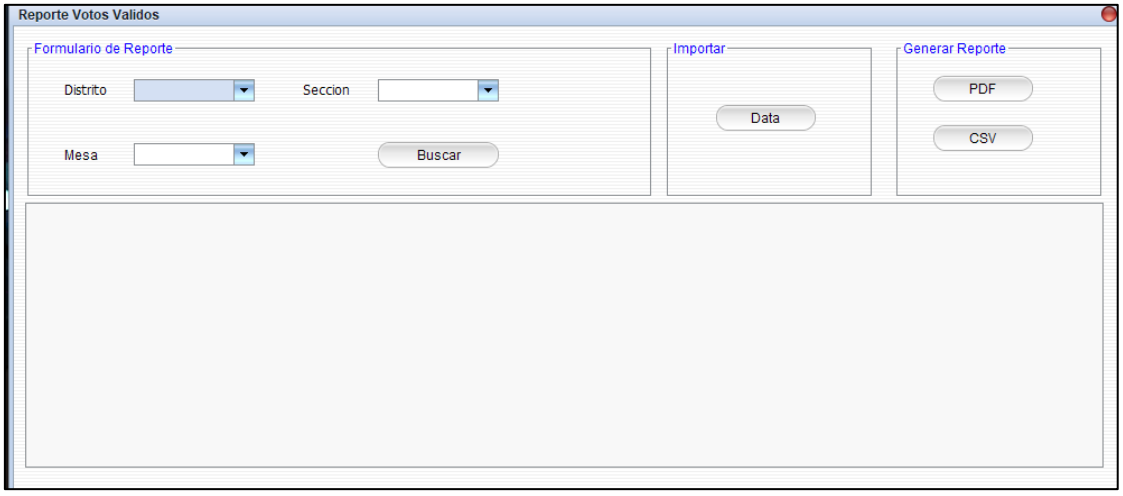
Consulta Votaciones



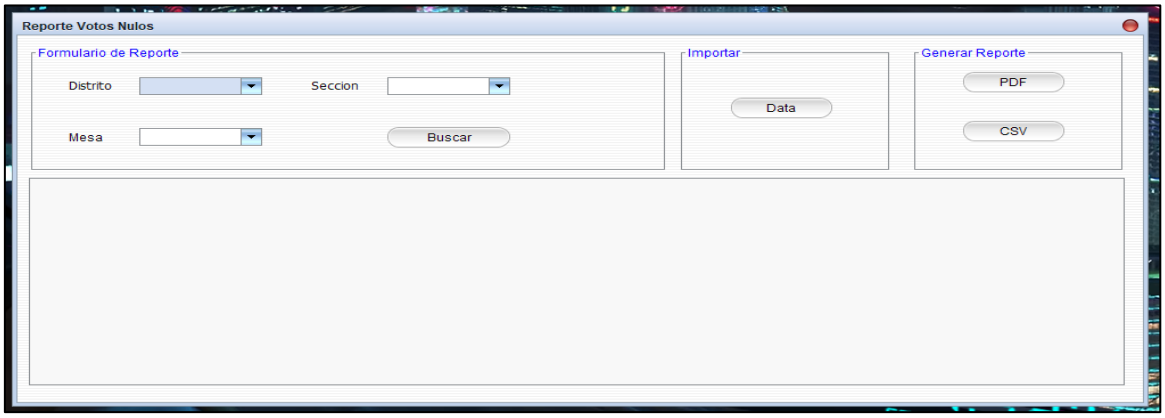
Registro de votos



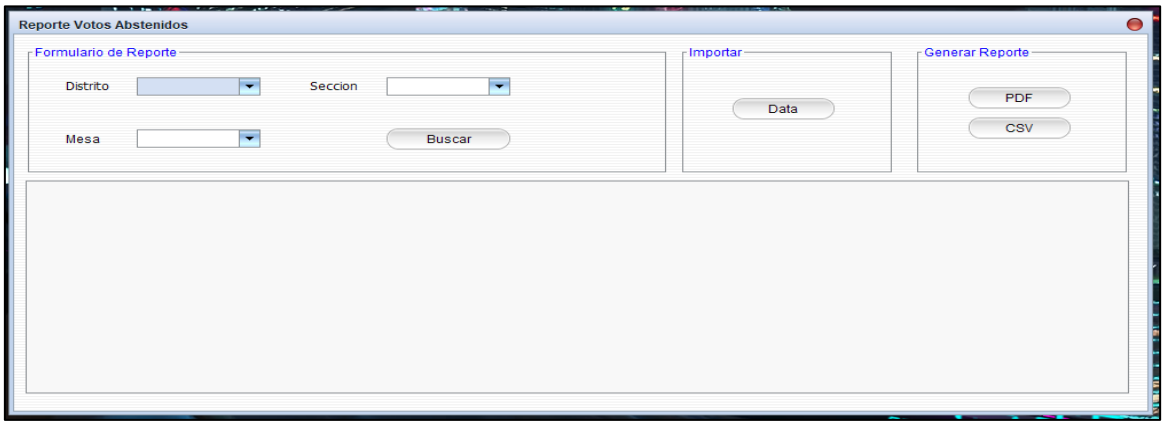
Reporte de votos validos



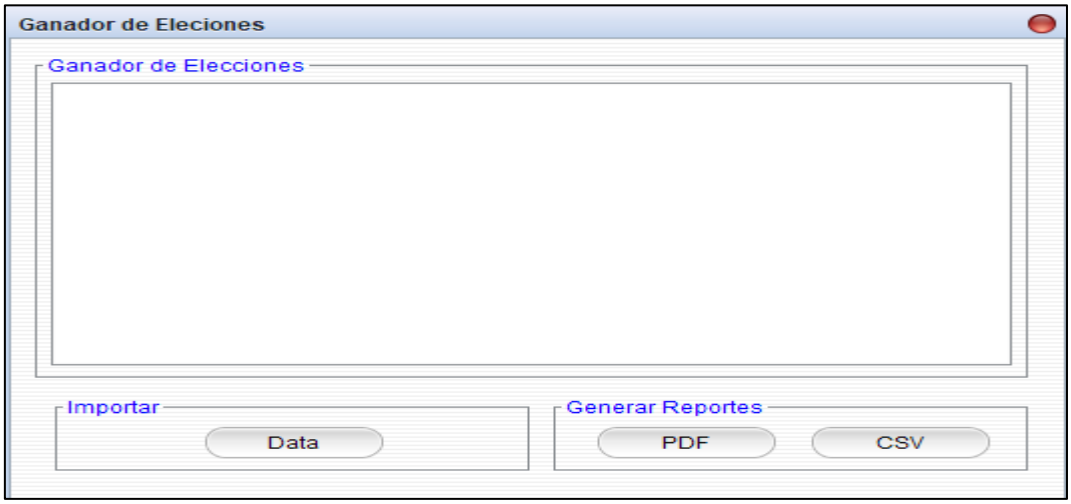
Reporte de votos nulos



Reporte de votos abstenidos



Generador de elecciones



3.1 CONCLUSIONES

En primer lugar, la automatización de los procesos electorales, desde el registro de votantes hasta la generación de resultados, es fundamental para mejorar la eficiencia, reducir los errores humanos y garantizar la precisión de los resultados. La implementación de un sistema informático que gestione los datos de manera estructurada y eficiente contribuye significativamente a agilizar los tiempos de procesamiento y a aumentar la transparencia en el proceso electoral.

En segundo lugar, el diseño de un sistema flexible que permita importar y exportar datos en formatos estándar (como CSV y PDF) facilita la interoperabilidad entre plataformas y la actualización constante de los registros. Esto también mejora la capacidad de verificación y auditoría, reduciendo la posibilidad de inconsistencias en los datos y fomentando la confianza pública en los resultados.

4.1 RECOMENDACIONES

Como primera recomendación, es recomendable continuar con la implementación de tecnologías de automatización en otros aspectos del proceso electoral, como la validación de identidades y la verificación de la integridad de los votos, para minimizar aún más el riesgo de fraude o manipulación y garantizar la confianza en las elecciones.

Como segunda recomendación, Sería beneficioso realizar pruebas piloto periódicas del sistema en diferentes contextos y escenarios para identificar posibles mejoras en la interfaz de usuario y en el rendimiento del sistema, lo que permitirá una adaptación continua a las necesidades cambiantes del proceso electoral.

5.1 BIBLIOGRAFÍA

El País. (2024, diciembre 1). España, Elecciones Generales anticipadas, 23 Julio 2023. El País. Recuperado de https://elpais.com/espana/elecciones/generales/

OSCE. (2024, diciembre 1). Informe final: Elecciones Generales anticipadas, 23 Julio 2023. OSCE. Recuperado de https://www.osce.org/es/odihr/elections/spain/564919

Unión Europea. (2016, julio 21). Informe final de la misión de observación electoral: Elecciones Generales, Perú 2016. Unión Europea. Recuperado de https://www.eeas.europa.eu/sites/default/files/moeue_peru_informe_final_210716_es_0.pdf

Parlamento Europeo. (2023, septiembre 25). Informe sobre las elecciones europeas de 2024. Parlamento Europeo. Recuperado de https://www.europarl.europa.eu/doceo/document/A-9-2023-0332_ES.html

6.1 HISTORIAL DE VERSIONES

Fecha de Elaboración	Versión	Elaborado por	Descripción	Revisado por	Fecha de Revisión
15-10-2024	1.0	Equipo	Creación del documento		
20-11-2024	2.0	Equipo	Documentación de problemática y objetivos		
31-11-2024	3.0	Equipo	Documentación estructura y clases		